Guia Prático Completo — PIBITI 2026–2027

Detecção de Deepfakes de Áudio em Português Brasileiro por Decomposição Wavelet Packet e Descritores Estatísticos do Contorno de Onda das Sub-bandas

Bolsista: Thiago Mateus Schiavi

Orientador: Prof. Dr. Alex Marino Gonçalves de Almeida

Fatec Ourinhos

Programa PIBITI-CNPq (Projeto 2521)

Período: 01/09/2026 a 31/08/2027

Setembro de 2026

Sumário

I Fundamentos e Ambiente **6**

1 VISÃO GERAL DO PROJETO **7**
1.1 Objetivo do projeto . . . 7
1.2 O que são deepfakes de áudio? . . . 7
1.3 A abordagem wavelet packet . . . 7
1.4 Mapa dos 12 meses . . . 8
1.5 Núcleo vs Expansão . . . 8
1.6 Corpora PT-BR . . . 9
1.7 Métricas principais . . . 9
1.8 Boas práticas transversais . . . 9

2 MÊS 1 — FUNDAMENTAÇÃO E AMBIENTAÇÃO (set/2026) **11**
2.1 Semana 1: Panorama do problema . . . 11
2.1.1 O que é EER? . . . 11
2.1.2 Criar repositório Git . . . 12
2.2 Semana 2: Wavelets e Wavelet Packet . . . 13
2.2.1 O que é uma wavelet? . . . 13
2.2.2 DWT vs WPT . . . 13
2.2.3 Por que WPT para deepfakes? . . . 13
2.2.4 Família Daubechies . . . 13
2.3 Semana 3: Ambiente Python . . . 14
2.3.1 Instalação do ambiente . . . 14
2.3.2 Verificação . . . 14
2.3.3 Inventário dos corpora PT-BR . . . 14
2.4 Semana 4: Fichamento . . . 15
2.4.1 Usando Zotero . . . 15
2.4.2 Pergunta de pesquisa . . . 15

3 MÊS 2 — PARTIÇÃO E TEORIA WPT (out/2026) **17**
3.1 Partição dos dados . . . 17
3.1.1 O que é uma partição? . . . 17
3.1.2 Split por locutor . . . 17
3.1.3 Arquivo de partição . . . 18
3.1.4 Validação da partição . . . 19
3.1.5 Padronização do áudio (16 kHz, mono) . . . 20
3.2 Teoria WPT aprofundada . . . 21
3.2.1 Nível L e número de sub-bandas . . . 21
3.2.2 Wavelets-mãe para o projeto . . . 21

3.2.3 Reconstrução de sub-bandas no domínio do tempo 21
3.2.4 Entropia e seleção de base 22
3.3 Exercícios do Mês 2 22
3.4 Desafios do Mês 2 23

4 MÊS 3 — WPT-PILOTO E RECONSTRUÇÃO (nov/2026) 24
4.1 Referencial Teórico (versão 1) 24
4.2 Decomposição WPT 24
4.3 Reconstrução completa 25
4.4 Reconstrução por sub-banda 26
4.5 Validação da reconstrução 26
4.6 Bateria de testes com sinais sintéticos 27
4.7 Conservação de energia 28
4.8 Padding e bordas 28
4.9 Comprimento das sub-bandas 28
4.10 Script completo de validação 28
4.11 Exercícios e desafios 29

II Extração de Descritores 31

5 MÊS 4 — CALIBRAÇÃO E PRIMEIROS DESCRITORES (dez/2026) 32
5.1 Calibração do nível de decomposição 32
5.1.1 Por que calibrar? 32
5.1.2 Comparação $L = 4$ vs $L = 5$ 32
5.1.3 Comparação de wavelets 33
5.2 Primeiros descritores: Energia e RMS 33
5.2.1 O que são descritores? 33
5.2.2 Energia 33
5.2.3 RMS (Root Mean Square) 34
5.2.4 Calcular para todas as sub-bandas 34
5.3 ZCR (Taxa de Cruzamento por Zero) 34
5.3.1 O que é ZCR? 34
5.4 Combinando descritores 35
5.5 Envelope via Transformada de Hilbert 35
5.5.1 O que é o envelope? 35
5.5.2 Descritores do envelope 36
5.6 Documentação dos parâmetros 36
5.7 Exercícios e desafios 37

6 MÊS 5 — EXTRAÇÃO COMPLETA DOS DESCRITORES (jan/2027) 39
6.1 Conjunto completo de descritores por sub-banda 39
6.1.1 Entropia 39
6.1.2 Fator de forma 40
6.1.3 Fator de crista 40
6.2 Extrator completo 40
6.3 Gerar matriz de features 41
6.4 Salvando e carregando features 42
6.5 Análise exploratória 43

6.5.1 Sanity checks . . . 43
6.5.2 PCA (visualização) . . . 43
6.6 Exercícios e desafios . . . 44

III Treino e Avaliação do Núcleo 45

7 MÊS 6 — TREINO DO NÚCLEO + BIBLIOTECA v0 + RELATÓRIO PARCIAL (fev/2027) 46
7.1 Visão geral: o que fazer neste mês . . . 46
7.2 GMM (Gaussian Mixture Model) . . . 47
7.2.1 O que é um GMM? . . . 47
7.2.2 Implementação . . . 47
7.2.3 Score por razão de verossimilhança . . . 48
7.3 XGBoost . . . 49
7.3.1 O que é XGBoost? . . . 49
7.3.2 Implementação . . . 49
7.4 Métricas: EER e min-DCF . . . 50
7.4.1 Calcular EER . . . 50
7.4.2 Calcular min-DCF . . . 50
7.5 Comparação GMM vs XGBoost . . . 51
7.6 Custo em CPU . . . 51
7.7 Biblioteca de extração v0 . . . 52
7.8 Relatório parcial . . . 52
7.9 Exercícios e desafios . . . 53

IV Expansão 55

8 MÊS 7 — EXPANSÃO: SVM/RF, db8/sym8, L=5 (mar/2027) 56
8.1 Objetivo da expansão . . . 56
8.2 SVM (Support Vector Machine) . . . 56
8.2.1 O que é SVM? . . . 56
8.2.2 Implementação . . . 57
8.2.3 Busca de hiperparâmetros . . . 57
8.3 Random Forest . . . 58
8.3.1 O que é Random Forest? . . . 58
8.3.2 Implementação . . . 58
8.4 Novas wavelets: db8 e sym8 . . . 59
8.4.1 Comparação e Extração . . . 59
8.5 Nível de Decomposição L=5 . . . 60
8.6 Matriz de configuração . . . 60
8.7 Registrando experimentos . . . 60
8.8 Exercícios e Desafios . . . 61

9 MÊS 8 — DEMAIS CORPORA + CUSTO EM CPU (abr/2027) 63
9.1 Incorporar novos corpora . . . 63
9.2 Medir o custo em CPU . . . 64
9.2.1 Tempo de decomposição e extração por áudio . . . 64

9.2.2 Tempo de treino e inferência dos modelos . . . 65
9.2.3 Tabela de custos (Ilustrativa) . . . 65
9.3 Ajuste de escopo . . . 66
9.4 Exercícios . . . 66

V Avaliação e Consolidação 68

10 MÊS 9 — AVALIAÇÃO COMPLETA + CONTRIBUIÇÃO DE SUB-BANDAS (mai/2027) 69
10.1 Avaliação de Todas as Combinações . . . 69
10.1.1 Matriz de Resultados . . . 69
10.1.2 EER e min-DCF in-domain para cada combinação . . . 70
10.2 Análise de Contribuição de Sub-bandas . . . 71
10.2.1 Feature Importance (XGBoost) . . . 71
10.2.2 Permutation Importance . . . 72
10.2.3 Atenção às Altas Frequências . . . 72
10.3 Figuras Finais . . . 72
10.3.1 EER por Sub-banda / Nível / Wavelet . . . 72
10.3.2 Curva DET / ROC . . . 73
10.4 Exercícios e Desafios . . . 74

11 MÊS 10 — BIBLIOTECA CONSOLIDADA + REPOSITÓRIO + REDAÇÃO (jun/2027) 75
11.1 Consolidar a Biblioteca . . . 75
11.1.1 Estrutura Final . . . 75
11.1.2 O arquivo `pyproject.toml` . . . 76
11.1.3 Testes Unitários . . . 76
11.2 Organizar o Repositório . . . 77
11.2.1 Estrutura Completa do Projeto de Pesquisa . . . 77
11.2.2 README.md Completo . . . 78
11.3 Rascunho do Artigo . . . 78
11.4 Exercícios e Desafios . . . 78

VI Finalização 80

12 MÊS 11 — SUBMISSÃO DO ARTIGO + FINALIZAÇÃO DO ARTEFATO (jul/2027) 81
12.1 Revisão final do artigo . . . 81
12.1.1 Checklist de revisão . . . 81
12.1.2 Template de tabela de resultados . . . 82
12.2 Eventos alvo . . . 82
12.3 Congelar a biblioteca e o repositório . . . 83
12.3.1 Versionamento final . . . 83
12.3.2 LICENSE . . . 83
12.3.3 CITATION.cff . . . 84
12.3.4 DOI via Zenodo (opcional) . . . 84
12.4 Licenças dos corpora e LGPD . . . 84

12.5 Exercícios 85

13 MÊS 12 — RELATÓRIO FINAL + ENCERRAMENTO (ago/2027) 87
13.1 Relatório final 87
13.1.1 Estrutura do relatório 87
13.1.2 Compilar resultados finais 88
13.1.3 Script para gerar todas as figuras finais 88
13.2 Apresentação / Pôster 89
13.2.1 Estrutura da apresentação 89
13.3 Entregas finais 89
13.4 Trabalhos futuros 90

VII Apêndices 91

14 CHECKLIST GERAL DO PROJETO 92

15 BOAS PRÁTICAS TRANSVERSAIS 94

16 GLOSSÁRIO 95

17 BIBLIOGRAFIA RECOMENDADA 97

Parte I

Fundamentos e Ambiente

Capítulo 1

VISÃO GERAL DO PROJETO

1.1 Objetivo do projeto

Bem-vindo ao guia prático do projeto PIBITI 2026-2027! O objetivo central da nossa pesquisa é desenvolver e validar um método de detecção de deepfakes de áudio (falas sintéticas ou clonadas) focados no idioma Português Brasileiro.

Para isso, empregaremos a **Decomposição Wavelet Packet (WPT)** em conjunto com **descritores estatísticos do contorno de onda** das sub-bandas geradas. Estes descritores formarão um vetor de características (*features*) que será submetido a algoritmos clássicos de aprendizado de máquina.

O produto final e principal da nossa pesquisa será uma **biblioteca de extração de características reutilizável**, focada em ser eficiente (rodando primariamente em CPU) e eficaz na detecção de *spoofs*.

1.2 O que são deepfakes de áudio?

Em termos simples, os *deepfakes de áudio* são falas sintéticas geradas por modelos de Inteligência Artificial. Isso abrange dois grandes grupos:

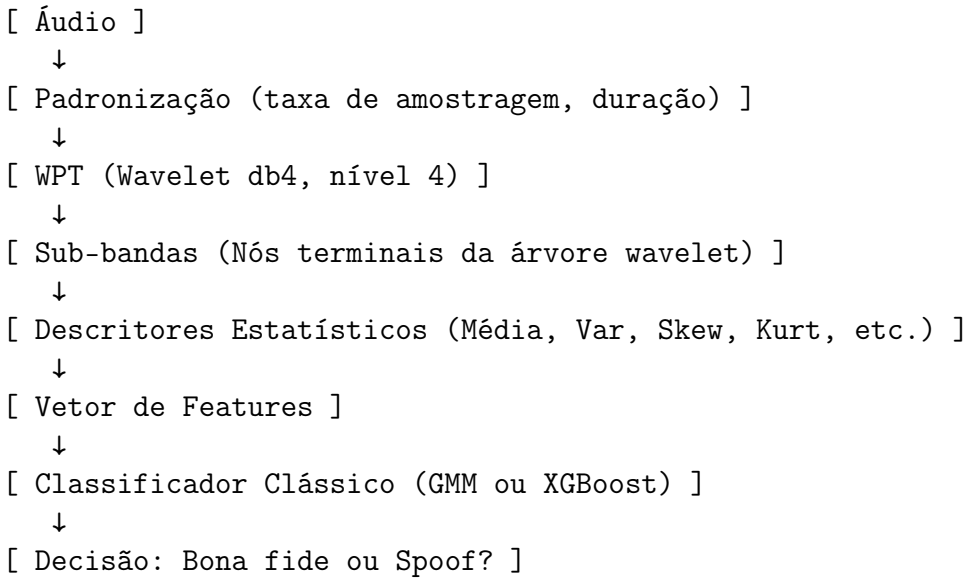
- **TTS (Text-to-Speech):** Geração de fala a partir de texto (vozes artificiais).
- **VC (Voice Conversion):** Conversão de voz, onde a fala de um locutor é transformada para soar como a de outro (clonagem de voz).

Na área de *Anti-spoofing* de voz, usamos duas nomenclaturas chave:

- **Bona fide:** Fala real, autêntica, humana.
- **Spoof:** Fala falsa, sintética ou gerada por IA.

1.3 A abordagem wavelet packet

Diferente de métodos tradicionais como MFCCs ou Espectrogramas baseados em Fourier, a Transformada Wavelet permite analisar o sinal mantendo um balanço excepcional entre resoluções de tempo e frequência. O nosso *pipeline* seguirá este fluxo:



1.4 Mapa dos 12 meses

O nosso cronograma abrange um ano de pesquisa intensiva. A tabela abaixo detalha as metas mensais:

Mês	**Período**	**Foco**	**Marco/Entrega**
1	set/2026	Fundamentação + ambiente	Ambiente + repositório Git
2	out/2026	Partição + teoria WPT	Partição validada
3	nov/2026	WPT-piloto (reconstrução)	Decomposição WPT funcionando
4	dez/2026	Calibra nível/wavelet + descritores	Sub-bandas + 1os descritores
5	jan/2027	Extração completa descritores	Vetor de features do núcleo
6	fev/2027	Treino + biblioteca v0 + relatório parcial	Relatório parcial + núcleo
7	mar/2027	Expansão: SVM/RF, db8/sym8, L=5	Matriz ampliada
8	abr/2027	Demais corpora + custo CPU	Cobertura ampliada
9	mai/2027	Avaliação completa	Resultados + análise
10	jun/2027	Biblioteca + repositório + redação	Biblioteca + rascunho
11	jul/2027	Submissão artigo	Artigo submetido
12	ago/2027	Relatório final	Relatório final

1.5 Núcleo vs Expansão

Nossa estratégia divide o trabalho em duas frentes para garantir entregas palpáveis sem sacrificar o potencial investigativo:

- **NÚCLEO (Entrega Mínima):** Decomposição no nível $L = 4$, wavelet Daubechies 4 (*db4*), utilizando 2 corpora básicos. Classificadores base: GMM e XGBoost.
- **EXPANSÃO:** Explorar níveis mais profundos ($L = 5$), outras wavelets como *db8* ou *sym8*, testes com classificadores adicionais (SVM, Random Forest) e englobar um total de 4 corpora em português brasileiro.

1.6 Corpora PT-BR

Nosso foco é inédito em avaliar a língua portuguesa. Levantamos os seguintes bancos de dados:

- **BRSpeech-DF (Costa Ferro Filho et al., 2025):** Focado em síntese (TTS/VC) em PT-BR.
- **FakeBrAccent (Santos et al., 2025):** Concentrado em sotaques do Brasil em fala sintética.
- **Fake Voices (UNFAKE/XTTS/CETUC, 2024):** Vozes clonadas recentes via modelos como XTTS.
- **BraSpoof-50 (França & Almeida, 2026):** Nova coleção ampla focada em TTS brasileiro.

1.7 Métricas principais

Em tarefas de anti-spoofing, a simples *acurácia* falha em oferecer um panorama real, especialmente porque bases de dados de deepfakes costumam ser muito desbalanceadas (muito mais spoofs do que falas reais).

Usaremos principalmente duas métricas:

- **EER (Equal Error Rate):** O ponto de operação onde a taxa de falsas aceitações empata com a taxa de falsas rejeições.
- **min-DCF (Minimum Detection Cost Function):** Uma função de custo que penaliza erros de forma ponderada. Fixaremos parâmetros como $C_{miss} = 1$, $C_{fa} = 1$ e probabilidade de alvo $P_{tar} = 0.05$.

1.8 Boas práticas transversais

Ao longo dos 12 meses, **sempre** adotaremos estas regras:

- Validar a reconstrução WPT antes de extrair qualquer feature.
- Manter um arquivo de partição (*splits*) de treino/validação/teste único e imutável (*frozen*).
- Extrair features apenas uma vez e salvar em cache (arquivos `.npy` ou `.h5`).
- Medir o custo computacional (tempo de CPU/memória) desde o primeiro script.

- Versionar todo o código no Git.
- **Nunca** misturar dados de validação com teste.
- Priorizar EER e min-DCF nas otimizações.
- Realizar reuniões quinzenais com o orientador.
- Redigir o relatório parcial e final em paralelo à execução do projeto.

Capítulo 2

MÊS 1 — FUNDAMENTAÇÃO E AMBIENTAÇÃO (set/2026)

Marco / Entrega

Ambiente reprodutível + repositório versionado + fichamento e inventário iniciados.

2.1 Semana 1: Panorama do problema

Na primeira semana de setembro, iniciamos o aprofundamento na literatura base de detecção de deepfakes. Entender a distinção entre a fala autêntica (*bona fide*) e a sintética (*spoof*) é crucial. Protocolos como os da série de desafios **ASVspoof** ditarão a forma de padronizar nossos experimentos.

2.1.1 O que é EER?

Para ilustrar o conceito da EER (Equal Error Rate), observe o código Python abaixo que simula como a métrica é avaliada em diferentes limiares (*thresholds*):

```python
import numpy as np

# Simulação simples de EER
# FAR (False Acceptance Rate): aceitar spoof como real
# FRR (False Rejection Rate): rejeitar real como spoof
# EER: ponto onde FAR = FRR

# Exemplo com scores simulados
np.random.seed(42)
scores_reais = np.random.normal(0.7, 0.15, 100)  # reais: scores altos
scores_falsos = np.random.normal(0.3, 0.15, 100)  # falsos: scores baixos

print(f"Scores reais - média: {scores_reais.mean():.2f}, std: {scores_reais.std():.2f}")
print(f"Scores falsos - média: {scores_falsos.mean():.2f}, std: {scores_falsos.std():.2f}")
```

```python

# Para cada threshold, calcular FAR e FRR
thresholds = np.arange(0, 1, 0.01)
for th in [0.3, 0.5, 0.7]:
    far = np.mean(scores_falsos >= th)  # falsos aceitos
    frr = np.mean(scores_reais < th)    # reais rejeitados
    print(f"Threshold {th:.1f}: FAR={far:.2f}, FRR={frr:.2f}")
```

Saída esperada

```
Scores reais - média: 0.68, std: 0.14
Scores falsos - média: 0.28, std: 0.14
Threshold 0.3: FAR=0.46, FRR=0.00
Threshold 0.5: FAR=0.07, FRR=0.09
Threshold 0.7: FAR=0.00, FRR=0.55
```

A EER reside próximo ao threshold 0.5, onde FAR (0.07) e FRR (0.09) estão mais próximos.

2.1.2 Criar repositório Git

Um pilar da nossa pesquisa é a reprodutibilidade e rastreabilidade. Começamos a estruturação criando nosso ambiente Git:

```
mkdir pibiti-deepfake
cd pibiti-deepfake
git init
mkdir data splits src features results reports
```

A função de cada diretório:

- `data/`: Áudios originais (serão ignorados pelo git).
- `splits/`: Metadados e arquivos indicando quem é treino/teste.
- `src/`: Todo nosso código fonte Python.
- `features/`: Vetores extraídos (também ignorados).
- `results/`: Logs, EERs e plots gerados.
- `reports/`: Notas de estudo, LaTeX e PDFs.

O conteúdo inicial do arquivo `.gitignore` evita submetermos gigabytes de áudios ao Github:

```
data/
features/*.npy
*.wav
__pycache__/
venv/
.DS_Store
```

2.2 Semana 2: Wavelets e Wavelet Packet

2.2.1 O que é uma wavelet?

Uma *wavelet* é intuitivamente uma "pequena onda". Diferente das senoides do Fourier, que duram do $-\infty$ ao $+\infty$, as wavelets são localizadas no tempo. Elas são usadas como um padrão ou "molde"para analisar as componentes do nosso sinal de áudio, rastreando descontinuidades sutis deixadas pelos vocoders sintéticos.

2.2.2 DWT vs WPT

- **DWT (Discrete Wavelet Transform):** A cada nível, decompõe apenas a aproximação (baixas frequências). Gera uma árvore assimétrica, detalhando mais os graves e menos os agudos.
- **WPT (Wavelet Packet Transform):** Decompõe TANTO a aproximação QUANTO os detalhes. Gera uma árvore binária simétrica completa.

A WPT entrega uma resolução de frequência muito mais uniforme, o que é fundamental no nosso cenário.

2.2.3 Por que WPT para deepfakes?

Artefatos de síntese muitas vezes residem nas mais variadas faixas de frequência, muito comumente nos sub-bandas de altas frequências devido à limitações dos filtros de *vocoding* usados por modelos de IA. A DWT ignora refinamentos em altas frequências. A WPT inspeciona todas as bandas de forma nivelada, capturando o fantasma sintético onde quer que ele esteja escondido.

2.2.4 Família Daubechies

Utilizaremos fortemente as wavelets da família Daubechies, desenhadas por Ingrid Daubechies. O código a seguir explora a nossa escolha padrão do projeto (db4):

```python
import pywt

# Listar wavelets Daubechies
print("Wavelets Daubechies:")
for w in pywt.wavelist('db'):
    wav = pywt.Wavelet(w)
    print(f"  {w:6s} - filtro: {wav.dec_len} coeficientes")
```

```python
# Propriedades da db4 (nossa wavelet principal)
wav = pywt.Wavelet('db4')
print(f"Nome: {wav.name}")
print(f"Família: {wav.family_name}")
print(f"Comprimento do filtro: {wav.dec_len}")
print(f"Ortogonal: {wav.orthogonal}")
print(f"Simetria: {wav.symmetry}")
```

Saída esperada

Nome: db4
Família: Daubechies
Comprimento do filtro: 8
Ortogonal: True
Simetria: asymmetric

2.3 Semana 3: Ambiente Python

2.3.1 Instalação do ambiente

No Windows, abra o PowerShell e execute:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

O arquivo `requirements.txt` deve conter as pedras fundamentais da nossa biblioteca:

```
numpy
scipy
PyWavelets
soundfile
librosa
scikit-learn
xgboost
matplotlib
```

2.3.2 Verificação

Crie um script simples para validar a stack inteira:

```
import numpy, scipy, pywt, soundfile, librosa, sklearn, xgboost
bibs = {"NumPy": numpy, "SciPy": scipy, "PyWavelets": pywt,
        "SoundFile": soundfile, "Librosa": librosa,
        "Scikit-learn": sklearn, "XGBoost": xgboost}
for nome, mod in bibs.items():
    print(f"{nome:15s} {mod.__version__}")
print("Ambiente OK!")
```

2.3.3 Inventário dos corpora PT-BR

Organizar o que testaremos desde o dia um é fundamental.

```
# Template para inventário de corpora
corpora = [
    {"nome": "BRSpeech-DF", "ref": "Costa Ferro Filho et al., 2025",
     "tipo": "TTS/VC", "horas_aprox": "?", "status": "verificar"},
    {"nome": "FakeBrAccent", "ref": "Santos et al., 2025",
```

```
     "tipo": "TTS", "horas_aprox": "?", "status": "verificar"},
    {"nome": "Fake Voices", "ref": "UNFAKE, 2024",
     "tipo": "XTTS/CETUC", "horas_aprox": "?", "status": "verificar
        "},
    {"nome": "BraSpoof-50", "ref": "França & Almeida, 2026",
     "tipo": "TTS", "horas_aprox": "?", "status": "verificar"},
]

print(f"{'Corpus':20s} {'Referência':35s} {'Tipo':10s} {'Status'}")
print("-" * 75)
for c in corpora:
    print(f"{c['nome']:20s} {c['ref']:35s} {c['tipo']:10s} {c['
       status']}")
```

2.4 Semana 4: Fichamento

2.4.1 Usando Zotero

A organização das referências bibliográficas não deve ser subestimada. Instale o Zotero e sua extensão para navegador. Sempre que encontrar um artigo (no Google Scholar ou IEEE Xplore) relevante, adicione ao Zotero, que extrairá os metadados (autores, ano, volume, DOI) e permitirá exportá-los perfeitamente no formato `.bib` para as nossas futuras submissões de artigos.

2.4.2 Pergunta de pesquisa

No fim do primeiro mês, temos nossa questão norteadora bem estabelecida, um norte absoluto:

Atenção

"Descritores estatísticos do contorno de onda de sub-bandas WPT, combinados com classificadores clássicos (GMM/XGBoost), detectam fala sintética em português brasileiro com bom desempenho (EER competitivo) e baixo custo computacional (CPU only)?"

Leituras do Mês

Leituras fundamentais para fechar o Mês 1:

- Daubechies, I. (1990) - The wavelet transform, time-frequency localization and signal analysis.
- Mallat, S. (2009) - A Wavelet Tour of Signal Processing (Capítulos sobre wavelets e wavelet packets).
- Coifman, R. R. & Wickerhauser, M. V. (1992) - Entropy-based algorithms for best basis selection.
- Wang, X. et al. (2020) - ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech.

Checklist de progresso (Mês 1):

☐ Repositório Git criado

☐ Estrutura de pastas definida

☐ Ambiente Python funcionando

☐ Bibliotecas instaladas

☐ Corpora inventariados

☐ Wavelets/WPT estudadas

☐ Fichamento iniciado (Zotero)

☐ Pergunta de pesquisa escrita

Capítulo 3

MÊS 2 — PARTIÇÃO E TEORIA WPT (out/2026)

Marco / Entrega

Arquivo de partição validado (ou gerado) + fichamento teórico da WPT consolidado.

Neste segundo mês, entraremos em duas frentes fundamentais: a organização metodológica dos nossos dados e o aprofundamento teórico na Transformada Wavelet Packet (WPT). Sem uma partição correta, nossos resultados perdem validade científica; sem uma base teórica sólida, a extração de características vira uma "caixa preta".

3.1 Partição dos dados

3.1.1 O que é uma partição?

Em aprendizado de máquina, nunca testamos nosso modelo com os mesmos dados usados para treiná-lo. Se fizéssemos isso, o modelo poderia simplesmente "decorar" os áudios, falhando miseravelmente ao encontrar novas amostras (fenômeno conhecido como *overfitting*). Por isso, dividimos nossos dados em três conjuntos (*splits*):

- **Treino (*train*):** É a maior porção (geralmente 60 a 80%). Usado para o modelo aprender os padrões que diferenciam vozes genuínas de *deepfakes*.
- **Validação (*dev* ou *validation*):** Usado durante o treinamento para ajustar hiperparâmetros e monitorar se o modelo está generalizando bem.
- **Teste (*test* ou *eval*):** O conjunto intocável. Usado apenas **uma vez**, no final do experimento, para avaliar o desempenho real e definitivo do modelo.

3.1.2 Split por locutor

Na detecção de *deepfakes* de áudio, a regra de ouro é: **o mesmo locutor não pode aparecer em conjuntos diferentes**.

Por que isso importa? Imagine que a voz do Locutor A está no treino. O modelo aprende as características vocais específicas dele. Se o Locutor A aparecer no conjunto de

teste, o modelo pode acertar não porque identificou o *deepfake*, mas porque reconheceu o locutor e memorizou como a voz dele soa quando é sintetizada no seu conjunto de treino. Para provar que seu detector de fato aprendeu a diferença entre áudio real e gerado, ele precisa ser testado com locutores **nunca antes vistos**.

```python
import numpy as np
from collections import defaultdict

# Simulação de partição por locutor
np.random.seed(42)

# Simular 50 locutores
locutores = [f"spk_{i:03d}" for i in range(50)]
np.random.shuffle(locutores)

# Dividir: 60% train, 20% dev, 20% test
n_train = int(0.6 * len(locutores))
n_dev = int(0.2 * len(locutores))

train_spk = locutores[:n_train]
dev_spk = locutores[n_train:n_train+n_dev]
test_spk = locutores[n_train+n_dev:]

print(f"Train: {len(train_spk)} locutores")
print(f"Dev:   {len(dev_spk)} locutores")
print(f"Test:  {len(test_spk)} locutores")

# Verificar: nenhum locutor em mais de um conjunto
assert len(set(train_spk) & set(dev_spk)) == 0
assert len(set(train_spk) & set(test_spk)) == 0
assert len(set(dev_spk) & set(test_spk)) == 0
print("Sem vazamento de locutor!")
```

Saída esperada

Train: 30 locutores Dev: 10 locutores Test: 10 locutores Sem vazamento de locutor!

3.1.3 Arquivo de partição

Para organizar essa divisão, usamos um arquivo delimitado, tipicamente um CSV ou TSV, que mapeia cada arquivo de áudio para seu respectivo locutor, rótulo (*bonafide* ou *spoof*) e *split*.

```python
import csv
import os

# Criar arquivo de partição
def criar_particao(caminho, audios):
    os.makedirs(os.path.dirname(caminho), exist_ok=True)
    with open(caminho, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
```

```
        writer.writerow(['arquivo', 'locutor', 'rotulo', 'split'])
        for audio in audios:
            writer.writerow(audio)
    print(f"Partição salva em {caminho}")

# Exemplo
audios = [
    ['audio_001.wav', 'spk_001', 'bonafide', 'train'],
    ['audio_002.wav', 'spk_001', 'spoof', 'train'],
    ['audio_003.wav', 'spk_025', 'bonafide', 'dev'],
    ['audio_004.wav', 'spk_040', 'spoof', 'test'],
]

criar_particao('splits/particao.csv', audios)
```

Atenção

O arquivo de partição é ÚNICO e CONGELADO. Nunca reparticione os dados após congelar o split. Isso é essencial para garantir a comparabilidade justa entre os diferentes experimentos que você fará no futuro.

3.1.4 Validação da partição

Erros acontecem. Antes de iniciar qualquer extração de características, devemos escrever um script que prove matematicamente que nossa partição não possui vazamentos de locutores e mantém o equilíbrio esperado das classes.

```
import csv
from collections import Counter

def validar_particao(caminho):
    with open(caminho, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        dados = list(reader)

    # Contar por split e rótulo
    splits = Counter(d['split'] for d in dados)
    rotulos = Counter(d['rotulo'] for d in dados)

    # Verificar vazamento de locutor
    locutores_por_split = {}
    for d in dados:
        split = d['split']
        if split not in locutores_por_split:
            locutores_por_split[split] = set()
        locutores_por_split[split].add(d['locutor'])

    print("Distribuição por split:")
    for split, count in splits.items():
        print(f"  {split}: {count} áudios")

```

```
    print(f"\nRótulos: {dict(rotulos)}")

    # Checar vazamento
    splits_list = list(locutores_por_split.keys())
    for i in range(len(splits_list)):
        for j in range(i+1, len(splits_list)):
            inter = locutores_por_split[splits_list[i]] &
               locutores_por_split[splits_list[j]]
            if inter:
                print(f"VAZAMENTO entre {splits_list[i]} e {
                   splits_list[j]}: {inter}")
            else:
                print(f"OK: sem vazamento entre {splits_list[i]} e
                   {splits_list[j]}")

validar_particao('splits/particao.csv')
```

3.1.5 Padronização do áudio (16 kHz, mono)

Ao mesclar diferentes *datasets*, muitas vezes encontramos variações na taxa de amostragem (8 kHz, 16 kHz, 44.1 kHz, 48 kHz) e no número de canais (mono, estéreo). O ideal é uniformizar os dados antes da análise. No contexto de voz, 16 kHz mono é o padrão da indústria.

```
import soundfile as sf
import librosa
import numpy as np
import os

def padronizar_audio(caminho_entrada, caminho_saida, sr_alvo=16000)
   :
    audio, sr = sf.read(caminho_entrada)

    # Converter para mono se for estéreo
    if audio.ndim == 2:
        audio = np.mean(audio, axis=1)

    # Reamostrar se a taxa for diferente da alvo
    if sr != sr_alvo:
        audio = librosa.resample(audio, orig_sr=sr, target_sr=
           sr_alvo)
        sr = sr_alvo

    sf.write(caminho_saida, audio, sr)
    return len(audio), sr
```

3.2 Teoria WPT aprofundada

3.2.1 Nível L e número de sub-bandas

Ao decompor um sinal usando a Wavelet Packet Transform (WPT), a cada nível L, tanto a aproximação (baixas frequências) quanto os detalhes (altas frequências) são divididos novamente. Assim, o número de sub-bandas no nível L é 2^L.

Para um áudio amostrado a 16.000 Hz, a frequência máxima representável é a de Nyquist ($f_s/2 = 8.000$ Hz). A largura de banda teórica de cada sub-banda no nível L é $(f_s/2)/2^L$.

Nível (L)	Qtd. Sub-bandas (2^L)	Resolução por banda (Hz)
1	2	4000.00
2	4	2000.00
3	8	1000.00
4	16	500.00
5	32	250.00
6	64	125.00

Tabela 3.1: Resolução de frequência da WPT para $f_s = 16.000$ Hz.

3.2.2 Wavelets-mãe para o projeto

A escolha da wavelet-mãe influencia a capacidade de detectar características abruptas (bom para sinais transitórios) ou variações suaves. Daubechies (db) e Symlets (sym) são muito utilizadas.

```
import pywt

for nome in ['db4', 'db8', 'sym8']:
    w = pywt.Wavelet(nome)
    print(f"{nome:6s}: filtro={w.dec_len:2d}, ortogonal={w.
        orthogonal}, simetria={w.symmetry}")
```

Saída esperada

db4 : filtro= 8, ortogonal=True, simetria=asymmetric db8 : filtro=16, ortogonal=True, simetria=asymmetric sym8 : filtro=16, ortogonal=True, simetria=near symmetric

3.2.3 Reconstrução de sub-bandas no domínio do tempo

Uma vantagem poderosa da WPT sobre outras transformadas é que os coeficientes mantêm forte relação com o domínio temporal. Você pode zerar os coeficientes indesejados e aplicar a transformada inversa (IDWT) para recuperar a porção do áudio correspondente apenas àquela faixa de frequência, possibilitando análises no tempo e na frequência de forma simultânea.

3.2.4 Entropia e seleção de base

Introduzido por Coifman e Wickerhauser, o algoritmo de *Best Basis* tenta encontrar a representação mais esparsa do sinal. Ao invés de usar todas as sub-bandas do nível mais baixo, calcula-se a entropia da informação. Uma árvore WPT pode ser podada onde os nós "filhos"têm maior entropia que o nó "pai".

```
import numpy as np
import pywt

def shannon_entropy(coefs):
    coefs_norm = coefs ** 2 / np.sum(coefs ** 2)
    coefs_norm = coefs_norm[coefs_norm > 0]  # evitar log(0)
    return -np.sum(coefs_norm * np.log(coefs_norm))

# Teste
fs = 16000
t = np.arange(0, 1, 1/fs)
audio = np.sin(2 * np.pi * 440 * t)

wp = pywt.WaveletPacket(data=audio, wavelet='db4', mode='symmetric', maxlevel=4)
nodes = wp.get_level(4, order='freq')

for i, node in enumerate(nodes):
    if np.sum(node.data**2) > 1e-10:
        ent = shannon_entropy(node.data)
        print(f"Sub-banda {i:2d} ({node.path}): entropia = {ent:.4f}")
    else:
        print(f"Sub-banda {i:2d} ({node.path}): energia ~0 (entropia indefinida)")
```

3.3 Exercícios do Mês 2

Exercício

1. Modifique a função de particionamento para criar também um conjunto de treino pequeno, com apenas 10% dos locutores do conjunto original, mantendo a proporção de rótulos.
2. Escreva uma função que leia o CSV da partição e crie dicionários mapeando caminhos de áudio para rótulos.
3. Usando a biblioteca `soundfile`, leia um áudio curto, extraia seus primeiros 5 segundos e salve-o.
4. Pesquise e compare a resposta em frequência dos filtros *db4* e *sym4*.
5. Implemente uma função que calcule a energia total de um sinal de áudio lido com numpy.

3.4 Desafios do Mês 2

Desafio

1. Escreva um script para detectar e registrar arquivos corrompidos em um dataset antes da partição.
2. Projete uma função *stratified_split* garantindo balanço exato de gêneros (masculino/feminino), caso os metadados existam.
3. Crie um script de visualização da distribuição de classes e splits através da biblioteca `matplotlib`.
4. Pesquise as wavelets de Coifman (*coif*). Teste a propriedade *vanishing moments*.
5. Desenvolva uma versão própria da entropia baseada na norma L1, em vez de Shannon, e teste.

Leituras do Mês

Referências importantes: Coifman & Wickerhauser (1992) - "Entropy-based algorithms for best basis selection". Lee et al. (2019) - Documentação e aplicações da `PyWavelets`. Artigos originais de datasets como ASVspoof 2019/2021 ou FOR.

Checklist - Mês 2

- ☐ Partição criada ou adotada (caso utilize datasets padrão como ASVspoof)
- ☐ Validação robusta de vazamento de locutor e de balanceamento
- ☐ Arquivo CSV/TSV congelado e versionado (não se mexe mais nele!)
- ☐ Níveis da WPT e impacto da resolução matemática estudados
- ☐ Entropia para seleção de base compreendida
- ☐ Fichamento teórico consolidado na documentação do projeto

Capítulo 4

MÊS 3 — WPT-PILOTO E RECONSTRUÇÃO (nov/2026)

Marco / Entrega

Referencial escrito + decomposição e reconstrução WPT validadas (piloto db4).

Atenção

ATENÇÃO: Este é o PRINCIPAL RISCO TÉCNICO do projeto. Não há biblioteca pronta de extração direta de características WPT como temos no domínio cepstral (como o MFCC ou LFCC no librosa). Teste absolutamente tudo com sinais sintéticos rigorosos antes de encostar nos corpora de voz.

4.1 Referencial Teórico (versão 1)

O mês 3 marca a redação da primeira versão do referencial teórico. Seu texto deve conter os seguintes tópicos base:

- **Wavelets e WPT:** Histórico, diferença para Fourier (STFT), janelamento variável, e por que a WPT é útil ao extrair características de altas frequências (em comparação à DWT convencional).
- **Descritores de contorno (Estatísticas):** Por que extraímos estatísticas (média, variância, curtose, *skewness*, energia, entropia) do sinal reconstruído. A WPT devolve coeficientes; transformar isso num vetor fixo é essencial.
- **Corpora e Métricas de Avaliação:** Detalhar os datasets (ASVspoof, por exemplo) e métricas, principalmente a *Equal Error Rate* (EER) e o *min t-DCF*.

4.2 Decomposição WPT

A decomposição real começa agora. Usaremos a biblioteca `PyWavelets` para realizar a transformada e acessar os nós correspondentes às bandas de frequência desejadas.

```python
import numpy as np
import pywt

def decompor_wpt(audio, wavelet='db4', modo='symmetric', nivel=4):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet,
                            mode=modo, maxlevel=nivel)
    # A ordem 'freq' garante que os nós saiam da menor para maior frequência
    nodes = wp.get_level(nivel, order='freq')

    subbandas = []
    for node in nodes:
        subbandas.append({
            'path': node.path,
            'dados': node.data,
            'amostras': len(node.data),
            'energia': np.sum(node.data ** 2)
        })

    return wp, subbandas

# Exemplo
fs = 16000
t = np.arange(0, 1, 1/fs)
audio = np.sin(2 * np.pi * 440 * t)

wp, subbandas = decompor_wpt(audio)

print(f"Sub-bandas: {len(subbandas)}")
for i, sb in enumerate(subbandas):
    print(f"  {i:2d} | {sb['path']:8s} | {sb['amostras']:6d} amostras | energia: {sb['energia']:.4f}")
```

4.3 Reconstrução completa

O primeiro teste de sanidade é garantir que o processo matemático é reversível. Ao reconstruir a árvore completa sem alterações, devemos obter o sinal original.

```python
def reconstruir_wpt(wp, tamanho_original):
    # update=False para evitar alterar os dados originais no nó
    reconstruido = wp.reconstruct(update=False)
    return reconstruido[:tamanho_original]

reconstruido = reconstruir_wpt(wp, len(audio))
erro = np.max(np.abs(audio - reconstruido))
print(f"Erro máximo de reconstrução: {erro:.2e}")
```

4.4 Reconstrução por sub-banda

Em vez de usar todos os coeficientes, o nosso projeto precisa isolar as sub-bandas e extrair características temporais de cada banda. Para isso, criamos uma árvore nula (zeros) e injetamos apenas os coeficientes de **um único nó** antes de inverter a transformada.

```
def reconstruir_subbanda(node, wavelet='db4', modo='symmetric',
   nivel=4, tamanho_original=None):
    wp_temp = pywt.WaveletPacket(data=None, wavelet=wavelet, mode=
       modo, maxlevel=nivel)
    wp_temp[node.path] = node.data
    rec = wp_temp.reconstruct(update=False)
    if tamanho_original:
        rec = rec[:tamanho_original]
    return rec
```

4.5 Validação da reconstrução

Precisamos quantificar com exatidão a perda energética e o desvio entre os sinais. A função de validação abaixo serve como o auditor dos testes que você rodará em seguida.

```
def validar_reconstrucao(original, reconstruido):
    n = min(len(original), len(reconstruido))
    orig = original[:n]
    rec = reconstruido[:n]

    erro_max = np.max(np.abs(orig - rec))
    erro_medio = np.mean(np.abs(orig - rec))
    mse = np.mean((orig - rec) ** 2)
    energia_orig = np.sum(orig ** 2)
    energia_rec = np.sum(rec ** 2)

    print("=" * 50)
    print("RELATÓRIO DE VALIDAÇÃO")
    print("=" * 50)
    print(f"Amostras:           {n}")
    print(f"Erro máximo:        {erro_max:.2e}")
    print(f"Erro médio:         {erro_medio:.2e}")
    print(f"MSE:                {mse:.2e}")
    print(f"Energia original:   {energia_orig:.6f}")
    print(f"Energia reconstruída: {energia_rec:.6f}")
    print(f"Diferença energia:  {abs(energia_orig - energia_rec):.2
       e}")

    if erro_max < 1e-10:
        print("STATUS: Reconstrução PERFEITA")
    elif erro_max < 1e-6:
        print("STATUS: Reconstrução EXCELENTE")
    else:
        print("STATUS: VERIFICAR - erro maior que esperado")
    print("=" * 50)
```

```
    return erro_max, mse
```

4.6 Bateria de testes com sinais sintéticos

Não confie no código cega e diretamente em áudio real. Testaremos casos extremos conhecidos matematicamente.

```
# Preparando bateria de sinais sintéticos (fs=16000, 1s)
t = np.arange(0, 16000) / 16000.0

# 1. Seno Puro (440 Hz)
sig_sine = np.sin(2 * np.pi * 440 * t)

# 2. Dois senos (440 Hz + 4000 Hz)
sig_two_sine = np.sin(2 * np.pi * 440 * t) + 0.5 * np.sin(2 * np.pi
    * 4000 * t)

# 3. Ruído Branco (Gaussiano)
np.random.seed(42)
sig_noise = np.random.randn(len(t))

# 4. Seno + Ruído Branco
sig_sine_noise = sig_sine + 0.1 * sig_noise

# 5. Impulso (Kronecker Delta em t=0.5s)
sig_impulse = np.zeros(len(t))
sig_impulse[8000] = 1.0

# 6. Constante
sig_const = np.ones(len(t))

testes = [
    ("Seno Puro", sig_sine),
    ("Dois Senos", sig_two_sine),
    ("Ruído Branco", sig_noise),
    ("Seno + Ruído", sig_sine_noise),
    ("Impulso", sig_impulse),
    ("Constante", sig_const)
]

for nome, sinal in testes:
    print(f"\nTestando: {nome}")
    wp, subb = decompor_wpt(sinal)
    rec = reconstruir_wpt(wp, len(sinal))
    validar_reconstrucao(sinal, rec)
```

4.7 Conservação de energia

Pelo Teorema de Parseval (válido para wavelets ortogonais), a energia total de um sinal (soma de seus valores ao quadrado) deve ser perfeitamente dividida pelas sub-bandas do nível de decomposição, com exceção de erros de truncamento em precisão de ponto flutuante.

```
# Testando Parseval
energia_sinal = np.sum(sig_noise ** 2)
wp, _ = decompor_wpt(sig_noise)

energia_subbandas = sum(np.sum(n.data ** 2) for n in wp.get_level
    (4))

print(f"Energia do sinal:      {energia_sinal:.6f}")
print(f"Energia somada (WPT):  {energia_subbandas:.6f}")
print(f"Diferença relativa:    {abs(energia_sinal -
    energia_subbandas) / energia_sinal:.2e}")
```

4.8 Padding e bordas

Quando a janela de filtragem passa nas pontas do sinal, faltam valores no array de dados. Existem várias formas de lidar com isso (*padding mode*):

- **symmetric (sym):** Espelha o sinal na borda. Menor efeito de artefato repentino e, por isso, é a recomendação padrão.
- **zero (zpd):** Adiciona zeros além da borda. Gera artefatos de "saltos"(*artifacts*) se as pontas do sinal estiverem distantes do zero.
- **periodic (per):** O final do sinal é conectado ao começo. Ideal caso seu sinal seja puramente periódico no trecho recortado, senão cria frequências agudas e espúrias.

4.9 Comprimento das sub-bandas

Ao fazer as *downsamplings* iterativas nas divisões, cada nível reduz o tamanho das sub-bandas aproximadamente por dois. Entenda que, devido às margens computadas no filtro (tamanho do filtro e modo de *padding*), o comprimento exato dos coeficientes quase sempre é ligeiramente **maior** do que $(N/2^L)$. Isso é necessário para que a *Perfect Reconstruction* ocorra, evitando que pontos na ponta do sinal fiquem mal cobertos pelas convoluções.

4.10 Script completo de validação

Reúna todas as funções da etapa 4 em um arquivo único (por exemplo, `valida_wpt.py`). Ele será sua apólice de seguro contra *bugs* matemáticos e teóricos no seu framework de extração. O *script* deve reportar "Sucesso"quando passado por todos os cenários sem erro.

4.11 Exercícios e desafios

Exercício

1. Decomponha o sinal 'Seno Puro' até o nível 4 e visualize graficamente a energia espalhada em todas as 16 sub-bandas.
2. Com base no *padding*, gere a decomposição com 'zero' e 'symmetric' num mesmo sinal longo. Subtraia a matriz de nós e plot onde reside a maior diferença.
3. Descubra como usar pywt.Wavelet para imprimir as funções de escalonamento (*scaling function*) e a *wavelet* e exibi-las em gráficos.
4. Recorte do sinal 'Dois Senos' de forma que um pedaço tenha tamanho ímpar e o submeta a função `reconstruir_wpt`. Avalie as mudanças.
5. Crie um script *wrapper* para calcular a transformada pacote no sinal 'Impulso' e localize, via índice de maior energia temporal, qual sub-banda tem maior coeficiente.

Desafio

1. Utilize o *scipy.signal.welch* ou a *STFT* sobre o sinal reconstruído num único nó. Demonstre que a frequência dominando a porção do espectro bate precisamente com a banda teórica (Tabela do nível L=4).
2. Implemente na mão e mostre que a soma linear na dimensão temporal das saídas de `reconstruir_subbanda(node)` para todos os 2^L nós recompõe o sinal real perfeitamente. Dica: as wavelets no domínio temporal não requerem janelamentos de recombinação (*overlap-add*).
3. Experimente extrair métricas de variância de cada `node.data`. Construa um vetor *feature* das 16 variâncias das bandas e plote a evolução disso durante uma amostra de voz real.
4. Troque a wavelet-mãe (ex. *coif1*, *db2*) nas funções testadas. Registre numericamente o impacto na *Perfect Reconstruction*. Se houver erro, descreva o porquê.
5. Escreva no seu projeto de referencial um subcapítulo analisando a superioridade (ou a viabilidade) do cálculo temporal e estatístico da IDWT para detectar artefatos gerados nos limites do espectro de *deepfakes*.

Leituras do Mês

Referências importantes: Mallat, S. (2009) - "A Wavelet Tour of Signal Processing". (A Bíblia). Documentação online oficial do PyWavelets: `https://pywavelets.readthedocs.io/`. Daubechies, I. (1990) - "The wavelet transform, time-frequency localization and signal analysis".

Checklist - Mês 3

- ☐ Estudo sobre bases e reconstrução documentado
- ☐ Código-fonte para 'decompor_wpt' feito
- ☐ Código-fonte para 'reconstruir_subbanda' feito e livre de bugs
- ☐ Validação estatística/energética bem sucedida nos 6 cenários falsos

- ☐ Efeitos e nuances de tamanhos dos coeficientes de borda (*padding*) aprendidos
- ☐ Referencial teórico 1.0 no Overleaf em progressão

Parte II

Extração de Descritores

Capítulo 5

MÊS 4 — CALIBRAÇÃO E PRIMEIROS DESCRITORES (dez/2026)

Marco / Entrega

Nível/wavelet do núcleo definidos; sub-bandas reconstruídas + primeiros descritores (energia/RMS, ZCR).

5.1 Calibração do nível de decomposição

5.1.1 Por que calibrar?

A escolha do nível de decomposição L na Transformada em Pacotes de Wavelet (WPT) define a resolução em frequência que obteremos, mas com um custo temporal. Quando $L = 4$, geramos $2^4 = 16$ sub-bandas. Se avançarmos para $L = 5$, teremos $2^5 = 32$ sub-bandas.

Ter mais sub-bandas significa uma resolução de frequência mais fina, o que é excelente para capturar pequenas anomalias geradas por algoritmos de deepfake. No entanto, por causa do princípio da incerteza, o aumento da resolução em frequência diminui o tamanho dos segmentos no tempo (resolução temporal). Além disso, mais sub-bandas geram matrizes de características (features) maiores, demandando mais poder computacional e memória. Essa escolha é, portanto, um compromisso (trade-off) importante.

5.1.2 Comparação $L = 4$ vs $L = 5$

```
import numpy as np
import pywt
import time

fs = 16000
t = np.arange(0, 1, 1/fs)
audio = np.sin(2 * np.pi * 440 * t) + 0.5 * np.sin(2 * np.pi * 2000
    * t)

```

```
for nivel in [3, 4, 5]:
    inicio = time.time()
    wp = pywt.WaveletPacket(data=audio, wavelet='db4', mode='
        symmetric', maxlevel=nivel)
    nodes = wp.get_level(nivel, order='freq')
    tempo = time.time() - inicio

    n_sub = len(nodes)
    resolucao = (fs/2) / n_sub  # Hz por sub-banda
    amostras = len(nodes[0].data)

    print(f"Nível {nivel}: {n_sub:3d} sub-bandas | {resolucao:.0f}
        Hz/banda | {amostras} amostras/banda | {tempo:.4f}s")
```

5.1.3 Comparação de wavelets

```
for wav_nome in ['db4', 'db8', 'sym8']:
    inicio = time.time()
    wp = pywt.WaveletPacket(data=audio, wavelet=wav_nome, mode='
        symmetric', maxlevel=4)
    rec = wp.reconstruct(update=False)[:len(audio)]
    tempo = time.time() - inicio
    erro = np.max(np.abs(audio - rec))
    print(f"{wav_nome:6s}: erro={erro:.2e}, tempo={tempo:.4f}s")
```

Dica

Comece com $L = 4$, usando a wavelet `db4` (que será o nosso núcleo de trabalho inicial). Guarde a expansão para wavelets mais complexas (como `db8`, `sym8`) e o nível $L = 5$ para a etapa de experimentação no mês 7.

5.2 Primeiros descritores: Energia e RMS

5.2.1 O que são descritores?

Descritores (também conhecidos como *features* ou características) são valores numéricos que resumem propriedades essenciais de um sinal ou de uma sub-banda específica. Em vez de passarmos milhares de amostras cruas para o classificador, fornecemos apenas alguns números altamente informativos, facilitando a distinção entre um áudio genuíno e um deepfake.

5.2.2 Energia

A energia de um sinal em tempo discreto é a soma do quadrado de suas amostras.

```
def energia(sinal):
    return np.sum(sinal ** 2)
```

5.2.3 RMS (Root Mean Square)

O valor RMS indica o valor quadrático médio, uma forma comum de mensurar a amplitude efetiva do sinal.

```
def rms(sinal):
    return np.sqrt(np.mean(sinal ** 2))
```

5.2.4 Calcular para todas as sub-bandas

Ao extrair a energia e o RMS de todas as sub-bandas, criamos um vetor de features.

```
def extrair_energia_rms(audio, wavelet='db4', nivel=4):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode='
        symmetric', maxlevel=nivel)
    nodes = wp.get_level(nivel, order='freq')

    features = []
    for node in nodes:
        e = np.sum(node.data ** 2)
        r = np.sqrt(np.mean(node.data ** 2))
        features.extend([e, r])

    return np.array(features)

# Teste
fs = 16000
t = np.arange(0, 1, 1/fs)
audio = np.sin(2 * np.pi * 440 * t)

vetor = extrair_energia_rms(audio)
print(f"Vetor de features: {len(vetor)} valores")
print(f"(16 sub-bandas x 2 descritores = {16*2})")
print(f"Primeiros valores: {vetor[:6]}")
```

5.3 ZCR (Taxa de Cruzamento por Zero)

5.3.1 O que é ZCR?

A Taxa de Cruzamento por Zero (Zero-Crossing Rate - ZCR) conta quantas vezes o sinal muda de positivo para negativo (ou vice-versa) ao longo do tempo. Sinais com frequências mais altas tendem a oscilar mais rápido, cruzando o eixo zero mais vezes, resultando em um ZCR maior. No contexto da detecção de deepfakes, discrepâncias em altas frequências deixadas pelos geradores neurais muitas vezes alteram a ZCR.

```
def zcr(sinal):
    # Conta mudancas de sinal
    cruzes = np.sum(np.abs(np.diff(np.sign(sinal))) > 0)
    return cruzes / len(sinal)

# Teste com diferentes frequencias
```

```
fs = 16000
t = np.arange(0, 1, 1/fs)

for freq in [100, 440, 1000, 4000]:
    sinal = np.sin(2 * np.pi * freq * t)
    z = zcr(sinal)
    print(f"{freq:5d} Hz: ZCR = {z:.6f}")
```

5.4 Combinando descritores

```
def extrair_descritores_basicos(audio, wavelet='db4', nivel=4):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode='
        symmetric', maxlevel=nivel)
    nodes = wp.get_level(nivel, order='freq')

    features = []
    for node in nodes:
        dados = node.data
        e = np.sum(dados ** 2)
                                                    # energia
        r = np.sqrt(np.mean(dados ** 2))
                                              # RMS
        z = np.sum(np.abs(np.diff(np.sign(dados))) > 0) / len(dados
            )  # ZCR
        features.extend([e, r, z])

    return np.array(features)

vetor = extrair_descritores_basicos(audio)
print(f"Vetor: {len(vetor)} features (16 x 3 = {16*3})")
```

5.5 Envelope via Transformada de Hilbert

5.5.1 O que é o envelope?

O envelope traça o contorno de amplitude de um sinal. Ele suaviza as variações rápidas de frequência para focar nas modulações de longo prazo da energia sonora. Obtemos o envelope aplicando a Transformada de Hilbert, que gera o sinal analítico; a magnitude desse sinal é o nosso envelope.

```
from scipy.signal import hilbert

# Exemplo
fs = 16000
t = np.arange(0, 0.01, 1/fs)  # 10ms
sinal = np.sin(2 * np.pi * 440 * t) * (1 + 0.5 * np.sin(2 * np.pi *
    5 * t))

```

```python
# Envelope
sinal_analitico = hilbert(sinal)
envelope = np.abs(sinal_analitico)

print(f"Sinal: min={sinal.min():.4f}, max={sinal.max():.4f}")
print(f"Envelope: min={envelope.min():.4f}, max={envelope.max():.4f}")
```

5.5.2 Descritores do envelope

```python
import scipy.stats

def descritores_envelope(sinal):
    sinal_analitico = hilbert(sinal)
    env = np.abs(sinal_analitico)

    return {
        'env_media': np.mean(env),
        'env_desvio': np.std(env),
        'env_curtose': float(scipy.stats.kurtosis(env)),
    }
```

Atenção

O envelope vem da transformada de Hilbert (`scipy.signal.hilbert` → magnitude do sinal analítico). Extraia dele média, desvio padrão e curtose.

5.6 Documentação dos parâmetros

Para fins de reprodutibilidade científica, é vital manter a rastreabilidade dos experimentos.

```python
# SEMPRE documente os parametros usados
config = {
    'wavelet': 'db4',
    'nivel': 4,
    'modo_borda': 'symmetric',
    'sr': 16000,
    'descritores': ['energia', 'rms', 'zcr'],
}
print("Configuracao:")
for k, v in config.items():
    print(f"  {k}: {v}")
```

5.7 Exercícios e desafios

Exercício

Implemente uma função que exiba graficamente o sinal original e o seu envelope obtido por Hilbert.

Exercício

Altere o nível de decomposição para 5 no teste prático e registre quanto o vetor de descritores aumenta em tamanho.

Exercício

Calcule o tempo médio de execução para extrair `energia`, `RMS` e `ZCR` de 100 áudios aleatórios de 1 segundo.

Exercício

Crie um dicionário `config` que rastreie também a data do experimento e a versão do `pywt`.

Desafio

Compare a curtose do envelope de um ruído branco puro contra um sinal senoidal limpo. O que você observa?

Desafio

Implemente um teste automatizado simples que verifique se as *features* não contêm NaNs ou infinitos após a extração da ZCR.

Desafio

Descubra a frequência aproximada de um sinal sonoro baseando-se no valor de ZCR e compare com a frequência real em um ambiente controlado.

Desafio

Empregue paralelismo (ex: `multiprocessing`) para processar a extração básica em múltiplas *threads*.

Leituras do Mês

Para maior aprofundamento: Coifman & Wickerhauser (1992); documentação do módulo `scipy.signal.hilbert`; definições matemáticas de Fator de Forma (shape factor) e Fator de Crista (crest factor).

Checklist Mês 4:

- Definição empírica do nível $L = 4$ e wavelet `db4`.
- Implementação e teste dos descritores: energia, RMS e ZCR.

- Entendimento do Envelope de Hilbert.
- Sistema de rastreabilidade de configurações estabelecido.

Capítulo 6

MÊS 5 — EXTRAÇÃO COMPLETA DOS DESCRITORES (jan/2027)

Marco / Entrega

Extrator do contorno de onda completo + matriz de features do núcleo versionada.

6.1 Conjunto completo de descritores por sub-banda

Para nosso sistema de detecção de deepfakes, definimos um total de 9 descritores por sub-banda:

1. Energia
2. RMS
3. ZCR
4. Envelope - média
5. Envelope - desvio padrão
6. Envelope - curtose
7. Entropia
8. Fator de forma (shape factor)
9. Fator de crista (crest factor)

6.1.1 Entropia

A entropia de Shannon aplicada à distribuição de energia ajuda a entender quão "espalhada"ou "concentrada"está a energia na sub-banda. Sinais de voz reais e gerados artificialmente muitas vezes possuem distribuições de energia diferentes.

```python
def entropia_sinal(sinal):
    # Entropia de Shannon baseada na distribuicao de energia
    energia = sinal ** 2
    total = np.sum(energia)
    if total < 1e-10:
        return 0.0
    prob = energia / total
    prob = prob[prob > 0]
    return -np.sum(prob * np.log2(prob))
```

6.1.2 Fator de forma

```python
def fator_de_forma(sinal):
    # RMS / media do valor absoluto
    media_abs = np.mean(np.abs(sinal))
    if media_abs < 1e-10:
        return 0.0
    return rms(sinal) / media_abs
```

6.1.3 Fator de crista

```python
def fator_de_crista(sinal):
    # Pico / RMS
    r = rms(sinal)
    if r < 1e-10:
        return 0.0
    return np.max(np.abs(sinal)) / r
```

6.2 Extrator completo

```python
import numpy as np
import pywt
from scipy.signal import hilbert
from scipy.stats import kurtosis

def extrair_todos_descritores(audio, wavelet='db4', nivel=4, modo='
    symmetric'):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode=modo,
        maxlevel=nivel)
    nodes = wp.get_level(nivel, order='freq')

    todas_features = []
    nomes_features = []

    for i, node in enumerate(nodes):
        dados = node.data
```

```
15
16          # 1. Energia
17          e = np.sum(dados ** 2)
18          # 2. RMS
19          r = np.sqrt(np.mean(dados ** 2))
20          # 3. ZCR
21          z = np.sum(np.abs(np.diff(np.sign(dados))) > 0) / len(dados
                )
22          # 4-6. Envelope (Hilbert)
23          env = np.abs(hilbert(dados))
24          env_media = np.mean(env)
25          env_std = np.std(env)
26          env_kurt = float(kurtosis(env))
27          # 7. Entropia
28          ent = entropia_sinal(dados)
29          # 8. Fator de forma
30          media_abs = np.mean(np.abs(dados))
31          ff = r / media_abs if media_abs > 1e-10 else 0.0
32          # 9. Fator de crista
33          fc = np.max(np.abs(dados)) / r if r > 1e-10 else 0.0
34
35          feats = [e, r, z, env_media, env_std, env_kurt, ent, ff, fc
                ]
36          todas_features.extend(feats)
37
38          for nome in ['energia', 'rms', 'zcr', 'env_media', 'env_std
                ', 'env_kurt', 'entropia', 'fator_forma', 'fator_crista'
                ]:
39              nomes_features.append(f"sb{i:02d}_{nome}")
40
41      return np.array(todas_features), nomes_features
42
43  # Teste
44  fs = 16000
45  t = np.arange(0, 1, 1/fs)
46  audio = np.sin(2 * np.pi * 440 * t)
47
48  vetor, nomes = extrair_todos_descritores(audio)
49  print(f"Total de features: {len(vetor)}")
50  print(f"(16 sub-bandas x 9 descritores = {16*9})")
51  print(f"\nPrimeiros nomes: {nomes[:9]}")
52  print(f"Primeiros valores: {vetor[:9]}")
```

6.3 Gerar matriz de features

```
1  import os
2  import soundfile as sf
3  import librosa
4
5  def gerar_matriz_features(lista_audios, wavelet='db4', nivel=4):
```

```
    matriz = []
    rotulos = []

    for caminho, rotulo in lista_audios:
        audio, sr = sf.read(caminho)
        # Padronizar
        if audio.ndim == 2:
            audio = np.mean(audio, axis=1)
        if sr != 16000:
            audio = librosa.resample(audio, orig_sr=sr, target_sr
               =16000)

        vetor, _ = extrair_todos_descritores(audio, wavelet, nivel)
        matriz.append(vetor)
        rotulos.append(rotulo)

    return np.array(matriz), np.array(rotulos)

# Exemplo com sinais sinteticos
print("Gerando matriz de features (simulacao)...")
fs = 16000
t = np.arange(0, 1, 1/fs)

# Simular 10 audios "reais" e 10 "falsos"
matriz = []
rotulos = []

for i in range(20):
    if i < 10:  # "real"
        audio = np.sin(2 * np.pi * 440 * t) + np.random.randn(len(t
           )) * 0.05
        rotulos.append(0)  # bonafide
    else:  # "falso"
        audio = np.sin(2 * np.pi * 440 * t) + np.random.randn(len(t
           )) * 0.2
        rotulos.append(1)  # spoof

    vetor, _ = extrair_todos_descritores(audio)
    matriz.append(vetor)

X = np.array(matriz)
y = np.array(rotulos)
print(f"Matriz: {X.shape}")
print(f"Rotulos: {y.shape}")
print(f"Bonafide: {np.sum(y==0)}, Spoof: {np.sum(y==1)}")
```

6.4 Salvando e carregando features

```
# Salvar
np.save('features/X_nucleo_db4_L4.npy', X)
```

```
np.save('features/y_nucleo.npy', y)
print("Features salvas!")

# Carregar
X_carregado = np.load('features/X_nucleo_db4_L4.npy')
y_carregado = np.load('features/y_nucleo.npy')
print(f"Features carregadas: {X_carregado.shape}")
```

Dica

Extraia as features uma vez e salve em disco. Não re-extraia a cada experimento — isso economiza tempo e garante reprodutibilidade.

6.5 Análise exploratória

6.5.1 Sanity checks

Antes de treinar qualquer modelo, verifique a integridade dos seus dados.

```
# Verificar problemas
print(f"NaN: {np.isnan(X).sum()}")
print(f"Inf: {np.isinf(X).sum()}")
print(f"Zeros: {np.sum(X == 0)}")
print(f"Media: {X.mean():.4f}")
print(f"Std: {X.std():.4f}")
print(f"Min: {X.min():.4f}")
print(f"Max: {X.max():.4f}")
```

6.5.2 PCA (visualização)

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)

plt.figure(figsize=(8, 6))
plt.scatter(X_2d[y==0, 0], X_2d[y==0, 1], label='Bonafide', alpha
    =0.7)
plt.scatter(X_2d[y==1, 0], X_2d[y==1, 1], label='Spoof', alpha=0.7)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
plt.title('PCA - Features Wavelet')
plt.savefig('resultados/pca_features.png')
plt.show()

print(f"Variancia explicada: {pca.explained_variance_ratio_}")
```

6.6 Exercícios e desafios

Exercício

Modifique o extrator completo para suportar extração de *features* por blocos/frames em vez do áudio inteiro.

Exercício

Crie um teste usando `pytest` para a função `fator_de_crista` garantindo a tratativa de divisão por zero.

Exercício

Implemente a normalização (*StandardScaler*) dos dados logo após a criação da matriz `X`.

Exercício

Em vez do PCA clássico, utilize o t-SNE para visualizar a matriz sintética em 2D.

Desafio

Adicione dois descritores estatísticos novos, como *Skewness* e mediana do contorno de Hilbert.

Desafio

Simule *outliers* e observe como o "sanity check"reage aos novos valores *NaN*.

Desafio

Use a biblioteca `pandas` para estruturar os rótulos (`nomes_features`) em um *DataFrame* junto com as *features*.

Desafio

Investigue se o fator de forma tem maior variância que o fator de crista em um *dataset* real de sons do dia a dia.

Leituras do Mês

Explore a documentação do módulo `scipy.stats` para distribuições, a referência `numpy` para cálculos matriciais e faça uma revisão de descritores de sinais.

Checklist Mês 5:

- Implementação do extrator completo de 9 descritores.
- Geração da matriz de características (features).
- Métodos para salvar e recuperar as features otimizadas.
- Rotinas de *sanity check* e visualização exploratória elaboradas.

Parte III

Treino e Avaliação do Núcleo

Capítulo 7

MÊS 6 — TREINO DO NÚCLEO + BIBLIOTECA v0 + RELATÓRIO PARCIAL (fev/2027)

Marco / Entrega

Relatório parcial entregue + núcleo validado (EER/min-DCF/custo) + biblioteca de extração v0.

7.1 Visão geral: o que fazer neste mês

Chegamos a um dos marcos mais importantes do nosso projeto PIBITI! Após extrair descritores utilizando a Transformada Wavelet em Pacotes (WPT), nosso foco agora é usar essas características para treinar modelos de detecção e avaliar o desempenho do nosso núcleo computacional.

Neste mês de fevereiro, devemos alcançar os seguintes objetivos:

- Treinar classificadores GMM (Gaussian Mixture Model) e XGBoost com os descritores extraídos da WPT.
- Avaliar e comparar o desempenho dos modelos através das métricas EER e min-DCF.
- Analisar o custo computacional (tempo de CPU) das nossas funções.
- Empacotar o código em uma biblioteca de extração inicial (versão v0).
- Escrever e estruturar o relatório parcial exigido pela pró-reitoria de pesquisa.

Prepare-se, pois teremos muita codificação, validação rigorosa e a formalização dos nossos esforços até agora.

7.2 GMM (Gaussian Mixture Model)

7.2.1 O que é um GMM?

O *Gaussian Mixture Model* (Modelo de Mistura Gaussiana) é um modelo probabilístico que assume que os dados podem ser gerados a partir de uma mistura de distribuições gaussianas com parâmetros desconhecidos. Em térmos simples, ele agrupa os dados usando distribuições normais (aquela clássica curva em formato de sino) multivariadas.

Na detecção de *deepfakes*, nós criamos dois GMMs:

1. Um GMM treinado apenas com áudios autênticos (*bonafide*).
2. Um GMM treinado apenas com áudios falsos (*spoof*).

Na hora da inferência, um áudio desconhecido é avaliado pelos dois modelos. Calculamos a probabilidade do áudio pertencer a cada um e a decisão é feita pela **razão de verossimilhança** (*likelihood ratio*) entre as duas probabilidades.

7.2.2 Implementação

A biblioteca `scikit-learn` fornece uma implementação robusta de GMM. Vejamos como treinar os modelos em Python:

```
from sklearn.mixture import GaussianMixture
import numpy as np

# Dados simulados para reproducibilidade
np.random.seed(42)
fs = 16000
t = np.arange(0, 1, 1/fs)

# Simular as features (como se ja tivessem sido extraidas pela WPT)
n_features = 144  # Exemplo: 16 sub-bandas x 9 descritores
n_train = 200

# Gerar dados sinteticos para demonstracao
# Bonafide com media 1.0 e spoof com media 0.5
X_bonafide_train = np.random.randn(n_train, n_features) * 0.5 + 1.0
X_spoof_train = np.random.randn(n_train, n_features) * 0.8 + 0.5

# Treinar dois GMMs independentes
n_componentes = 16  # Numero de gaussianas misturadas

gmm_bonafide = GaussianMixture(
    n_components=n_componentes,
    covariance_type='diag', # Matriz diagonal e mais rapida e comum
        em audio
    max_iter=200,
    random_state=42
)

gmm_spoof = GaussianMixture(
```

```
    n_components=n_componentes,
    covariance_type='diag',
    max_iter=200,
    random_state=42
)

# Ajuste (fit) dos modelos
gmm_bonafide.fit(X_bonafide_train)
gmm_spoof.fit(X_spoof_train)
print("GMMs treinados com sucesso!")
```

Dica

O parâmetro `covariance_type='diag'` diz ao algoritmo que os descritores são estatisticamente independentes. Isso reduz drasticamente o número de parâmetros a estimar e evita *overfitting*, sendo amplamente usado na literatura de processamento de fala.

7.2.3 Score por razão de verossimilhança

Para testar um áudio, extraímos o escore (o log da razão) subtraindo os *log-likelihoods*:

```
def calcular_scores_gmm(gmm_bonafide, gmm_spoof, X_teste):
    """
    Calcula o score de verossimilhanca.
    Score > 0 sugere bonafide; Score < 0 sugere spoof.
    Para inverter (maior = mais chance de spoof), basta multiplicar por -1.
    """
    log_like_bonafide = gmm_bonafide.score_samples(X_teste)
    log_like_spoof = gmm_spoof.score_samples(X_teste)

    # Razao de verossimilhanca (em escala logaritmica vira subtracao)
    scores = log_like_bonafide - log_like_spoof
    return scores

# Gerando dados de teste
X_teste_real = np.random.randn(50, n_features) * 0.5 + 1.0
X_teste_falso = np.random.randn(50, n_features) * 0.8 + 0.5

# Obtendo os scores
scores_real = calcular_scores_gmm(gmm_bonafide, gmm_spoof, X_teste_real)
scores_falso = calcular_scores_gmm(gmm_bonafide, gmm_spoof, X_teste_falso)

print(f"Scores reais - media: {scores_real.mean():.2f}")
print(f"Scores falsos - media: {scores_falso.mean():.2f}")
```

7.3 XGBoost

7.3.1 O que é XGBoost?

O *eXtreme Gradient Boosting* (XGBoost) é um algoritmo de aprendizado de máquina baseado em árvores de decisão e *Gradient Boosting*. Diferente do *Random Forest* (que treina árvores em paralelo), o XGBoost constrói árvores sequencialmente, onde cada nova árvore tenta corrigir os erros das árvores anteriores.

Ele é incrivelmente poderoso, muito rápido e frequentemente é o estado da arte para dados tabulares, que é exatamente o formato dos descritores wavelet que extraímos!

7.3.2 Implementação

Diferente do GMM, nós treinaremos um único classificador XGBoost passando as amostras de treino de *bonafide* (como classe 0) e *spoof* (como classe 1):

```
import xgboost as xgb
from sklearn.model_selection import train_test_split

# Preparar dados: empilhar e criar as labels
X_train = np.vstack([X_bonafide_train, X_spoof_train])
y_train = np.array([0]*n_train + [1]*n_train)  # 0=bonafide, 1=
    spoof

# Configurar e treinar XGBoost
modelo_xgb = xgb.XGBClassifier(
    n_estimators=100,        # Numero de arvores
    max_depth=6,             # Profundidade maxima de cada arvore
    learning_rate=0.1,       # Taxa de aprendizado
    random_state=42,
    eval_metric='logloss',
    use_label_encoder=False # Evita um warning chato
)

modelo_xgb.fit(X_train, y_train)
print("XGBoost treinado com sucesso!")

# Para o teste, faremos o mesmo empilhamento
X_teste = np.vstack([X_teste_real, X_teste_falso])
y_teste = np.array([0]*50 + [1]*50)

# Ao inves de predict, usamos predict_proba para obter o score
# predict_proba retorna [prob_classe0, prob_classe1]
# Queremos a probabilidade de ser spoof (classe 1)
scores_xgb = modelo_xgb.predict_proba(X_teste)[:, 1]

print(f"Scores XGB - media bonafide: {scores_xgb[:50].mean():.4f}")
print(f"Scores XGB - media spoof: {scores_xgb[50:].mean():.4f}")
```

7.4 Métricas: EER e min-DCF

Com as predições em mãos, precisamos medir objetivamente quão bem nosso modelo funciona usando os padrões da biometria de voz: *Equal Error Rate* (EER) e *minimum Detection Cost Function* (min-DCF).

7.4.1 Calcular EER

O EER ocorre no limiar (threshold) em que a Taxa de Falsos Positivos (FPR) é igual à Taxa de Falsos Negativos (FNR = 1 - TPR).

```
from scipy.optimize import brentq
from scipy.interpolate import interp1d
from sklearn.metrics import roc_curve

def calcular_eer(y_verdadeiro, scores):
    """Calcula o EER usando interpolacao."""
    fpr, tpr, thresholds = roc_curve(y_verdadeiro, scores)
    fnr = 1 - tpr

    # EER: ponto onde FPR = FNR
    # Usa a funcao brentq para encontrar a raiz (diferenca zero)
    eer_threshold = brentq(lambda x: interp1d(fpr, fpr)(x) -
        interp1d(fpr, fnr)(x), 0, 1)
    eer = interp1d(fpr, fpr)(eer_threshold)

    return eer * 100  # Retorna em porcentagem (%)

# Calcular EER do XGBoost
eer_xgb = calcular_eer(y_teste, scores_xgb)
print(f"EER XGBoost: {eer_xgb:.2f}%")
```

Uma abordagem mais simples e menos rígida que também funciona bem na prática:

```
def calcular_eer_simples(y_verdadeiro, scores):
    """Abordagem simplificada de EER buscando a menor diferenca na
       curva."""
    fpr, tpr, _ = roc_curve(y_verdadeiro, scores)
    fnr = 1 - tpr

    # Encontrar o indice do ponto mais proximo de FPR == FNR
    idx = np.nanargmin(np.abs(fpr - fnr))
    eer = (fpr[idx] + fnr[idx]) / 2

    return eer * 100
```

7.4.2 Calcular min-DCF

O min-DCF aplica um custo para falsos alarmes (C_{fa}) e erros ou omissões (C_{miss}), juntamente com a probabilidade *a priori* do evento (P_{tar}).

```python
def calcular_min_dcf(y_verdadeiro, scores, p_tar=0.05, c_miss=1, c_fa=1):
    fpr, tpr, _ = roc_curve(y_verdadeiro, scores)
    fnr = 1 - tpr

    # Calculo do DCF para cada ponto de threshold
    dcf = c_miss * fnr * p_tar + c_fa * fpr * (1 - p_tar)

    # Normalizar dividindo pelo melhor sistema trivial (aceitar tudo ou recusar tudo)
    dcf_norm = dcf / min(c_miss * p_tar, c_fa * (1 - p_tar))

    min_dcf = np.min(dcf_norm)
    return min_dcf

min_dcf_xgb = calcular_min_dcf(y_teste, scores_xgb)
print(f"min-DCF XGBoost: {min_dcf_xgb:.4f}")
```

Atenção

Fixe o min-DCF com $C_{miss} = C_{fa} = 1$ e $P_{tar} = 0.05$ — estes são os mesmos valores que configuraremos na frente de *benchmark*. Isso garante consistência ao comparar com outros trabalhos na literatura do ASVspoof.

7.5 Comparação GMM vs XGBoost

Sempre apresente seus resultados de forma organizada, como em tabelas, para facilitar a leitura.

```python
# Invertemos o sinal dos scores do GMM para que maior signifique spoof
scores_gmm_teste = np.hstack([-scores_real, -scores_falso])
eer_gmm = calcular_eer(y_teste, scores_gmm_teste)
min_dcf_gmm = calcular_min_dcf(y_teste, scores_gmm_teste)

# Tabela de resultados no console
print(f"{'Classificador':15s} {'EER (%)':>10s} {'min-DCF':>10s}")
print("-" * 40)
print(f"{'GMM':15s} {eer_gmm:10.2f} {min_dcf_gmm:10.4f}")
print(f"{'XGBoost':15s} {eer_xgb:10.2f} {min_dcf_xgb:10.4f}")
```

7.6 Custo em CPU

Um sistema eficiente é tão importante quanto um sistema preciso, especialmente se o objetivo é rodar no *edge* ou em dispositivos embarcados. Vamos medir o tempo:

```python
import time

def medir_custo(func, *args, repeticoes=5):
```

```
    """Mede a media e o desvio padrao do tempo de execucao de uma
       funcao."""
    tempos = []
    for _ in range(repeticoes):
        inicio = time.time()
        resultado = func(*args)
        tempos.append(time.time() - inicio)
    return np.mean(tempos), np.std(tempos)

# Exemplos de uso de medicao:
# Suponha que temos o pipeline de extracao pronto:
# def pipeline_extracao(audio):
#     vetor, _ = extrair_todos_descritores(audio)
#     return vetor
#
# def pipeline_inferencia_xgb(modelo, X):
#     return modelo.predict_proba(X)
#
# tempo_ext, std_ext = medir_custo(pipeline_extracao, audio_exemplo
    )
# tempo_inf, std_inf = medir_custo(pipeline_inferencia_xgb,
    modelo_xgb, vetor_exemplo)
#
# print("Custos em CPU:")
# print(f"  Extracao: {tempo_ext:.4f} +/- {std_ext:.4f} s")
# print(f"  Inferencia XGB: {tempo_inf:.4f} +/- {std_inf:.4f} s")
```

7.7 Biblioteca de extração v0

No mês 6, não podemos manter os códigos espalhados em dezenas de Jupyter Notebooks confusos. É o momento de refatorar e organizar o código em módulos reutilizáveis. A estrutura do seu pacote `src` deve ficar assim:

```
src/
  __init__.py
  wpt.py          # Funcoes para decomposicao WPT e reconstrucao de banda
  descritores.py  # Calculos (energia, entropia, teager, cruzamento por zero, etc)
  extracao.py     # Unificacao: recebe audio e retorna o vetor de caracteristicas fina
  metricas.py     # Funcoes de avaliacao (EER, min-DCF)
  utils.py        # Funcoes auxiliares, carregamento de audio e plotagem
```

Essa estrutura limpa (*v0*) será muito útil para importar scripts de treino rapidamente e garantirá que os scripts da frente de *benchmark* do próximo trimestre rodem perfeitamente.

7.8 Relatório parcial

No meio do projeto PIBITI, é obrigatório enviar um relatório para a instituição. Siga este esboço (*template*) para escrevê-lo:

1. **Introdução:** Contextualize os deepfakes de voz e a necessidade de detecção rápida.
2. **Referencial teórico:** Resuma, de forma simples, o que é a Transformada Wavelet em Pacotes (WPT).
3. **Metodologia:** Descreva as etapas do sistema, desde a aquisição (sinal, pré-ênfase), passagem na WPT e os sub-descritores computados, além dos modelos de classificação usados (GMM e XGBoost).
4. **Resultados preliminares (núcleo):** Mostre os gráficos da árvore WPT extraída, as tabelas de comparação (GMM vs XGBoost) com EER e min-DCF, e apresente as métricas de tempo e custo de CPU medidos.
5. **Próximos passos:** Mencione que o próximo semestre focará na construção de *benchmarks* em grandes bases de dados, contêiner Docker e otimização em C++.

7.9 Exercícios e desafios

Exercício

Altere os hiperparâmetros do XGBoost (por exemplo, diminua o `learning_rate` para 0.01 e aumente o `n_estimators` para 500) e veja se há ganho no EER.

Exercício

Crie a métrica de Acurácia e a Matriz de Confusão usando o `scikit-learn` além do EER.

Exercício

Exporte o modelo treinado do XGBoost usando a função `modelo_xgb.save_model('modelo.json')` e recarregue-o.

Exercício

No arquivo `metricas.py`, implemente uma função para gerar o gráfico da Curva ROC usando o `matplotlib`.

Desafio

Tente treinar um modelo *Random Forest* (com `sklearn.ensemble.RandomForestClassifier`) e compare a velocidade de inferência contra o XGBoost. Qual consome menos CPU na função `medir_custo`?

Desafio

Otimização de Threshold: Crie um script que pegue o EER do treino para definir o limiar de decisão (ao invés de 0.5 fixo) e aplique nos dados de teste para relatar o EER calibrado.

Desafio

Faça uma busca em grade (*GridSearch*) para tentar encontrar o número ótimo de componentes `n_components` do GMM variando de 8 a 64.

Desafio

Estruture e versione sua biblioteca `src` no GitHub/GitLab com o relatório parcial em formato Markdown no `README.md`.

Leituras do Mês

Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD '16.
Reynolds, D. A. (2015). *Gaussian Mixture Models*. Encyclopedia of Biometrics.

Marco / Entrega

Parabéns! Você concluiu o Mês 6 do cronograma! Seu checklist: [X] GMM implementado. [X] XGBoost implementado. [X] Métricas EER e min-DCF calculadas. [X] Custo de CPU em inferência medido. [X] Código refatorado para `src` v0. [X] Relatório parcial escrito e entregue.

Parte IV

Expansão

Capítulo 8

MÊS 7 — EXPANSÃO: SVM/RF, db8/sym8, L=5 (mar/2027)

Marco / Entrega

Matriz de configuração ampliada (SVM/RF, db8/sym8, L=5).

8.1 Objetivo da expansão

Neste mês, expandiremos nossos experimentos para além do núcleo inicial (que utilizou a wavelet Daubechies 4 `db4`, nível $L = 4$, e os classificadores GMM e XGBoost). O objetivo agora é explorar novas configurações para verificar se conseguimos melhorar o desempenho da detecção sem aumentar desproporcionalmente o custo computacional. Exploraremos:

- **Novos classificadores:** Máquinas de Vetores de Suporte (SVM) e Random Forest (RF).
- **Novas wavelets-mãe:** Daubechies 8 (`db8`) e Symlet 8 (`sym8`).
- **Novo nível de decomposição:** $L = 5$, que irá gerar 32 sub-bandas.

8.2 SVM (Support Vector Machine)

8.2.1 O que é SVM?

A Máquina de Vetores de Suporte (SVM) é um algoritmo de aprendizado de máquina poderoso e versátil. A ideia principal por trás do SVM é encontrar o hiperplano ideal que melhor separa as classes de dados no espaço de características. Para problemas não linearmente separáveis (como frequentemente é o caso com características de áudio), o SVM utiliza o "truque do kernel"(kernel trick), mapeando os dados para um espaço de dimensões superiores onde a separação linear se torna possível. Utilizaremos comumente o kernel RBF (Radial Basis Function).

8.2.2 Implementação

Para implementar o SVM em Python, utilizaremos a biblioteca `scikit-learn`. É crucial lembrar que o SVM é sensível à escala dos dados, portanto, a padronização é uma etapa obrigatória.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

# IMPORTANTE: SVM precisa de padronizacao!
pipeline_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(
        kernel='rbf',
        probability=True,  # necessario para calcular os scores/
           probabilidades
        random_state=42
    ))
])

# Dados simulados para demonstracao
np.random.seed(42)
n_features = 144
n_train = 200
X_train = np.random.randn(n_train*2, n_features)
y_train = np.array([0]*n_train + [1]*n_train)

# Treinamento do modelo
pipeline_svm.fit(X_train, y_train)
print("SVM treinado com sucesso!")

# Obtendo scores para avaliacao
X_teste = np.random.randn(100, n_features)
y_teste = np.array([0]*50 + [1]*50)
scores_svm = pipeline_svm.predict_proba(X_teste)[:, 1]
```

Atenção

`StandardScaler` é ESSENCIAL para o SVM. Sem a padronização prévia, características com escalas maiores dominarão a função de distância do SVM, e o modelo não funcionará adequadamente. Sempre utilize `Pipeline` para garantir que o escalonamento seja feito corretamente, evitando vazamento de dados.

8.2.3 Busca de hiperparâmetros

Para o SVM, especialmente com kernel RBF, o ajuste de hiperparâmetros como `C` (regularização) e `gamma` (coeficiente do kernel) é vital.

```
from sklearn.model_selection import GridSearchCV

```

```
param_grid = {
    'svm__C': [0.1, 1, 10],
    'svm__gamma': ['scale', 'auto'],
}

grid = GridSearchCV(
    pipeline_svm,
    param_grid,
    cv=3,
    scoring='roc_auc',
    n_jobs=-1
)

grid.fit(X_train, y_train)
print(f"Melhor C: {grid.best_params_['svm__C']}")
print(f"Melhor gamma: {grid.best_params_['svm__gamma']}")
print(f"Melhor AUC: {grid.best_score_:.4f}")
```

Atenção

Não misture validação e teste. Use `GridSearchCV` apenas nos conjuntos de treino e desenvolvimento (train+dev). NUNCA o utilize nos dados de teste, sob risco de corromper a validade dos seus resultados.

8.3 Random Forest

8.3.1 O que é Random Forest?

O Random Forest (Floresta Aleatória) é um método de *ensemble learning* que opera construindo uma múltipla coleção de árvores de decisão durante o treinamento. A predição final é baseada na votação majoritária (para classificação) das árvores individuais. Ele é robusto contra *overfitting* e geralmente exige menos pr-́processamento e ajuste de hiperparâmetros do que o SVM.

8.3.2 Implementação

```
from sklearn.ensemble import RandomForestClassifier

modelo_rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42,
    n_jobs=-1
)

# O treinamento do RF e direto
modelo_rf.fit(X_train, y_train)

# Extracao de scores
```

```
scores_rf = modelo_rf.predict_proba(X_teste)[:, 1]

print("Random Forest treinado!")
print(f"Score medio para audios bonafide: {scores_rf[:50].mean():.4f}")
print(f"Score medio para audios spoof: {scores_rf[50:].mean():.4f}")
```

8.4 Novas wavelets: db8 e sym8

8.4.1 Comparação e Extração

Wavelets com ordem superior, como a `db8` e `sym8`, possuem filtros mais longos. Isso pode resultar em um isolamento de frequências mais refinado, porém ao custo de maior exigência computacional e menor resolução temporal devido ao suporte maior da wavelet.

```
import pywt
import time
import numpy as np

fs = 16000
t = np.arange(0, 1, 1/fs)
# Sinal sintético simples
audio = np.sin(2 * np.pi * 440 * t) + np.random.randn(len(t)) * 0.1

print(f"{'Wavelet':6s} | Nível | Sub-bandas | Features | Erro de Recon. | Tempo (s)")
print("-" * 75)

resultados = []
for wav in ['db4', 'db8', 'sym8']:
    for nivel in [4, 5]:
        inicio = time.time()

        # Decomposicao Wavelet Packet
        wp = pywt.WaveletPacket(data=audio, wavelet=wav, mode='symmetric', maxlevel=nivel)
        nodes = wp.get_level(nivel, order='freq')

        # Reconstrucao para checar erro
        rec = wp.reconstruct(update=False)[:len(audio)]
        tempo = time.time() - inicio

        erro = np.max(np.abs(audio - rec))
        n_sub = len(nodes)
        n_feat = n_sub * 9  # 9 descritores estatisticos por sub-banda

        print(f"{wav:6s} | L={nivel} | {n_sub:10d} | {n_feat:8d} | {erro:.2e}     | {tempo:.4f}")
```

8.5 Nível de Decomposição L=5

Expandir de $L = 4$ para $L = 5$ significa dobrar o número de sub-bandas e, consequentemente, o número de características (features) do nosso vetor.

```
# L=5 gera 32 sub-bandas -> 32 x 9 = 288 features
# vs L=4: 16 sub-bandas -> 16 x 9 = 144 features

fs = 16000
for nivel in [4, 5]:
    n_sub = 2**nivel
    n_feat = n_sub * 9
    resolucao = (fs/2) / n_sub
    print(f"L={nivel}: {n_sub:2d} sub-bandas, {n_feat:3d} features,
        {resolucao:.0f} Hz/banda")
```

8.6 Matriz de configuração

Nesta etapa, temos múltiplas variáveis em nossos experimentos. Vamos visualizar quantas combinações precisaremos rodar e avaliar.

```
import itertools

wavelets = ['db4', 'db8', 'sym8']
niveis = [4, 5]
classificadores = ['GMM', 'XGBoost', 'SVM', 'RF']

print(f"{'Wavelet':8s} {'Nível':>6s} {'Classificador':>15s} {'
   Features':>10s}")
print("-" * 45)
for wav, nivel, clf in itertools.product(wavelets, niveis,
   classificadores):
    n_feat = (2**nivel) * 9
    print(f"{wav:8s} {nivel:6d} {clf:>15s} {n_feat:10d}")

total = len(wavelets) * len(niveis) * len(classificadores)
print(f"\nTotal de configuracoes: {total}")
```

8.7 Registrando experimentos

Com tantas combinações, gerenciar os resultados manualmente é impossível. Precisamos de um script robusto para logar os resultados em um CSV.

```
import csv
import os
from datetime import datetime

def registrar_experimento(caminho, resultado):
    cabecalho = ['data', 'corpus', 'wavelet', 'nivel', '
       classificador',
```

```
                    'eer', 'min_dcf', 'tempo_extracao', 'tempo_treino',
                    'tempo_inferencia', 'observacoes']

    arquivo_existe = os.path.exists(caminho)
    with open(caminho, 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=cabecalho)
        if not arquivo_existe:
            writer.writeheader()

        resultado['data'] = datetime.now().isoformat()
        writer.writerow(resultado)
    print(f"Experimento registrado em {caminho}")

# Exemplo de uso:
# res = {'corpus': 'ASVspoof', 'wavelet': 'db8', 'nivel': 5, '
    classificador': 'SVM', 'eer': 0.05}
# registrar_experimento('logs_experimentos.csv', res)
```

8.8 Exercícios e Desafios

Exercício

Crie um script Python que rode todas as combinações de classificadores e wavelets definidas na matriz de configuração utilizando os dados limitados do corpus `ASVspoof2019` que você já extraiu nos mêses anteriores.

Exercício

Incorpore o método `registrar_experimento` ao seu script de treinamento e geração de *scores* para automatizar a documentação dos resultados.

Exercício

Adicione o cálculo da métrica minDCF ao loop principal de experimentos antes de registrar os resultados no arquivo CSV.

Exercício

Pesquise e descreva a diferença teórica entre as propriedades de fase das wavelets Daubechies (db) e Symlets (sym).

Desafio

O `GridSearchCV` testado usou poucos parâmetros. Implemente um `RandomizedSearchCV` tanto para o SVM quanto para o Random Forest, testando uma maior gama de hiperparâmetros no conjunto de validação, a fim de extrair a melhor performance de cada modelo.

Desafio

Realize a Análise de Componentes Principais (PCA) sobre as matrizes de características (com $L = 5$, total de 288 features) e verifique se a redução de dimensionalidade afeta negativamente o EER no modelo SVM.

Leituras do Mês

Para aprofundar, consulte:

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Mallat, S. (2009). *A Wavelet Tour of Signal Processing: The Sparse Way*. 3a ed. Academic Press.

Checklist do Mês 7:

Compreensão e implementação do SVM e padronização.

Compreensão e implementação de Random Forest.

Comparação experimental entre wavelets `db4`, `db8` e `sym8`.

Ampliação da extração de características para $L = 5$.

Arquitetura estabelecida para *logging* automático de múltiplo experimentos.

Capítulo 9

MÊS 8 — DEMAIS CORPORA + CUSTO EM CPU (abr/2027)

Marco / Entrega

Cobertura de corpora ampliada + custo em CPU medido e registrado.

9.1 Incorporar novos corpora

Após termos estabelecido nossa base experimental no corpus `ASVspoof`, chegou o momento de estender nossa avaliação para outros datasets, como o `FakeBrAccent` e o `Fake Voices` (e demais conjuntos de dados menores que caibam em nosso orçamento de memória e CPU). Avaliar os modelos em corpora variados garante que a detecção de *deepfakes* seja robusta e independente da língua ou sotaque.

```
import os
import csv
import soundfile as sf
import numpy as np
# Supondo que a funcao extrair_todos_descritores ja esta implementada
# e importada no seu escopo principal

# Template padrao para processar um novo corpus estruturado via CSV
def processar_corpus(nome_corpus, diretorio, particao_csv, wavelet='db4', nivel=4):
    # Carregar metadata e particoes do CSV
    with open(particao_csv, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        entradas = list(reader)

    resultados = {
        'train': {'X': [], 'y': []},
        'dev': {'X': [], 'y': []},
        'test': {'X': [], 'y': []}
    }

```

```
    for entrada in entradas:
        caminho = os.path.join(diretorio, entrada['arquivo'])
        split = entrada['split']  # 'train', 'dev' ou 'test'
        rotulo = 0 if entrada['rotulo'] == 'bonafide' else 1

        # Leitura e padronizacao basica do audio
        audio, sr = sf.read(caminho)

        if audio.ndim == 2:
            # Converter stereo para mono
            audio = np.mean(audio, axis=1)

        # Extracao
        vetor, _ = extrair_todos_descritores(audio, wavelet, nivel)
        resultados[split]['X'].append(vetor)
        resultados[split]['y'].append(rotulo)

    # Converter listas puras para numpy arrays (crucial para Scikit-learn)
    for split in resultados:
        resultados[split]['X'] = np.array(resultados[split]['X'])
        resultados[split]['y'] = np.array(resultados[split]['y'])
        print(f"{nome_corpus} - Parti\c{c}\~ao {split}: {resultados[split]['X'].shape}")

    return resultados
```

9.2 Medir o custo em CPU

O objetivo de utilizar a Packet Wavelet Transform (WPT) não é apenas obter boa acurácia, mas garantir que o sistema resultante seja leve o suficiente para ser executado em dispositivos com capacidades limitadas (borda/*edge devices*). Portanto, o *profiling* computacional (medição de tempo e recursos da CPU e Memória RAM) é tão importante quanto o valor de EER.

9.2.1 Tempo de decomposição e extração por áudio

Nesta avaliação usamos o método `tracemalloc` padrão do Python para avaliar o pico de memória (Peak Memory) alocada durante o processamento de um único trecho.

```
import time
import tracemalloc

def medir_custo_completo(audio, wavelet='db4', nivel=4, repeticoes=10):
    tempos = []

    # Aquecimento de cache e medicao de tempo
    for _ in range(repeticoes):
        inicio = time.time()
```

```
        vetor, _ = extrair_todos_descritores(audio, wavelet, nivel)
        tempos.append(time.time() - inicio)

    # Medicao de memoria
    tracemalloc.start()
    vetor, _ = extrair_todos_descritores(audio, wavelet, nivel)
    _, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    media_tempo = np.mean(tempos)
    std_tempo = np.std(tempos)

    print(f"Tempo medio: {media_tempo:.4f} +/- {std_tempo:.4f} s")
    print(f"Pico de memoria RAM: {pico / 1024:.1f} KB")

    return media_tempo, pico
```

9.2.2 Tempo de treino e inferência dos modelos

Diferentes algoritmos de *machine learning* têm custos drásticamente distintos na etapa de treinamento, que é *offline*, e na fase de inferência, que ocorre em tempo-real (*online*).

```
def medir_tempo_treino(modelo, X_train, y_train):
    inicio = time.time()
    modelo.fit(X_train, y_train)
    tempo_total = time.time() - inicio
    return tempo_total

def medir_tempo_inferencia(modelo, X_teste, repeticoes=10):
    tempos = []
    for _ in range(repeticoes):
        inicio = time.time()
        _ = modelo.predict_proba(X_teste)
        tempos.append(time.time() - inicio)

    return np.mean(tempos)
```

9.2.3 Tabela de custos (Ilustrativa)

Com os scripts acima em execução ao longo de todos os experimentos, ao final teremos capacidade de compilar um levantamento semelhante à tabela teórica abaixo:

```
print(f"{'Etapa':25s} {'Tempo (s)':>12s} {'Memoria (KB)':>14s}")
print("-" * 55)
print(f"{'Decomposicao WPT':25s} {'0.0234':>12s} {'128.5':>14s}")
print(f"{'Extracao descritores':25s} {'0.0456':>12s} {'256.0':>14s}
    ")
print(f"{'Treino XGBoost':25s} {'2.3400':>12s} {'8192.0':>14s}")
print(f"{'Inferencia XGBoost':25s} {'0.0012':>12s} {'512.0':>14s}")
```

Dica

O custo computacional em CPU e memória é um resultado central e norteador deste projeto. Registre todos estes dados nos seus logs CSV de experiências desde o princípio, associando-os com as diferentes combinações da matriz.

9.3 Ajuste de escopo

Neste mês, se o seu monitoramento computacional demonstrar que certas configurações propostas (ex: utilizar wavelet `sym8` com matriz $L = 5$ no classificador Random Forest com 500 árvores) extrapolam a viabilidade prática ou travam seu computador devido à explosão de consumo de RAM, é perfeitamente justificável realizar um ajuste de escopo e documentar as limitações. Este tipo de refinamento iterativo faz parte intrínseca do ciclo da pesquisa. Expanda seus corpora de análise de forma gradual!

9.4 Exercícios

Exercício

Crie a estrutura de metadados necessária para o `FakeBrAccent`, adequando o mapeamento de *labels* aos rótulos padrão de bonafide (0) e spoof (1). Processe um lote de validação utilizando o template providenciado.

Exercício

Adicione as chamadas das funções de monitoramento (`medir_tempo_treino` e `medir_tempo_inferencia`) dentro do seu script principal de matriz de classificação criado no mês 7.

Exercício

Incorpore um cálculo para avaliar qual configuração oferece a melhor relação custo-benefício (Trade-off): um *score* unificado considerando uma minimização conjunta de tempo de inferência e erro EER.

Exercício

Observe e anote quais bibliotecas, `scikit-learn` ou `xgboost`, entregam os tempos de inferência (para vetores extraídos em nível de $L = 4$) mais curtos na sua máquina.

Leituras do Mês

Para explorar sobre avaliações justas de performance, busque sobre: *Benchmarking best practices in machine learning*. Adicionalmente, documentações dos módulos padrões de Python, `time` e `tracemalloc`.

Checklist do Mês 8:

Preparação e adaptação dos scripts para receber corpora além do corpus principal.

Implementação dos métodos de cronometragem com a biblioteca `time`.

Implementação dos métodos de medição de alocação de memória com a biblioteca `tracemalloc`.

Geração das métricas unificadas incluindo EER e métricas de viabilidade.

Ajustes e refatorações de código baseados no gargalo encontrado nos experimentos.

Parte V

Avaliação e Consolidação

Capítulo 10

MÊS 9 — AVALIAÇÃO COMPLETA + CONTRIBUIÇÃO DE SUB-BANDAS (mai/2027)

Marco / Entrega

Avaliação completa e análise de contribuição de sub-bandas/descritores concluída.

10.1 Avaliação de Todas as Combinações

10.1.1 Matriz de Resultados

Nesta etapa, o objetivo ê automatizar a avaliação para gerar uma matriz de resultados completa abrangendo diferentes wavelets, níveis de decomposição e classificadores para cada corpus de áudio analisado.

```
import itertools
import numpy as np
import csv

# Configuracoes
corpora = ['BRSpeech-DF', 'BraSpoof-50', 'FakeBrAccent', '
    FakeVoices']
wavelets = ['db4', 'db8', 'sym8']
niveis = [4, 5]
classificadores = ['GMM', 'XGBoost', 'SVM', 'RF']

# Template para armazenar resultados
resultados = []

# Exemplo de como registrar
resultado = {
    'corpus': 'BRSpeech-DF',
    'wavelet': 'db4',
    'nivel': 4,
    'classificador': 'XGBoost',
```

```
    'eer': 5.23,
    'min_dcf': 0.1234,
    'tempo_extracao_por_audio': 0.045,
    'tempo_treino': 2.34,
    'tempo_inferencia_por_audio': 0.001,
}
resultados.append(resultado)

# Salvar todos os resultados em CSV
def salvar_resultados_csv(resultados, caminho):
    if not resultados:
        return
    with open(caminho, 'w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=resultados[0].keys())
        writer.writeheader()
        writer.writerows(resultados)
    print(f"Resultados salvos em {caminho}")
```

10.1.2 EER e min-DCF in-domain para cada combinação

O código a seguir mostra o pipeline básico para treinar e avaliar o desempenho utilizando diversas métricas de verificação biométrica, com foco principal em Equal Error Rate (EER) e minimum Detection Cost Function (min-DCF).

```
def avaliar_configuracao(X_train, y_train, X_test, y_test,
   classificador_nome):
    from sklearn.metrics import roc_curve
    import time

    if classificador_nome == 'XGBoost':
        import xgboost as xgb
        modelo = xgb.XGBClassifier(n_estimators=100, max_depth=6,
                                   learning_rate=0.1, random_state
                                      =42,
                                   eval_metric='logloss')
    elif classificador_nome == 'SVM':
        from sklearn.svm import SVC
        from sklearn.preprocessing import StandardScaler
        from sklearn.pipeline import Pipeline
        modelo = Pipeline([('scaler', StandardScaler()),
                           ('svm', SVC(probability=True,
                              random_state=42))])
    elif classificador_nome == 'RF':
        from sklearn.ensemble import RandomForestClassifier
        modelo = RandomForestClassifier(n_estimators=200,
           random_state=42)

    # Treinar
    inicio_treino = time.time()
    modelo.fit(X_train, y_train)
    tempo_treino = time.time() - inicio_treino
```

```
24
25      # Inferencia
26      inicio_inf = time.time()
27      if classificador_nome == 'XGBoost':
28          scores = modelo.predict_proba(X_test)[:, 1]
29      else:
30          scores = modelo.predict_proba(X_test)[:, 1]
31      tempo_inf = time.time() - inicio_inf
32
33      # Metricas (funcoes definidas nos capitulos anteriores)
34      eer = calcular_eer_simples(y_test, scores)
35      min_dcf = calcular_min_dcf(y_test, scores)
36
37      return eer, min_dcf, tempo_treino, tempo_inf
```

10.2 Análise de Contribuição de Sub-bandas

10.2.1 Feature Importance (XGBoost)

Saber interpretar o seu modelo é fundamental. Modelos baseados em árvores, como o XGBoost, permitem que você acesse a importância de cada feature facilmente, nos dizendo quais sub-bandas e descritores mais ajudam na detecção.

```
1  import xgboost as xgb
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Apos treinar o modelo:
6  # importancias = modelo_xgb.feature_importances_
7
8  # Simular importancias para ilustracao
9  np.random.seed(42)
10 n_features = 144  # 16 sub-bandas x 9 descritores
11 importancias = np.random.rand(n_features)
12
13 # Agregar por sub-banda (9 descritores por sub-banda)
14 importancia_por_subbanda = np.zeros(16)
15 for i in range(16):
16     importancia_por_subbanda[i] = np.sum(importancias[i*9:(i+1)*9])
17
18 # Normalizar
19 importancia_por_subbanda /= importancia_por_subbanda.sum()
20
21 print("Importancia por sub-banda:")
22 for i, imp in enumerate(importancia_por_subbanda):
23     barra = '#' * int(imp * 100)
24     print(f"  Sub-banda {i:2d}: {imp:.4f} {barra}")
25
26 # Quais sub-bandas mais discriminam?
27 top_3 = np.argsort(importancia_por_subbanda)[::-1][:3]
```

```
print(f"\nTop 3 sub-bandas: {top_3}")
print("(Altas frequencias tendem a discriminar mais - artefatos de sintese)")
```

10.2.2 Permutation Importance

Outra estratégia, mais robusta e agnóstica em relação ao classificador, ê a *Permutation Importance*. Ela funciona baralhando as entradas de uma *feature* e observando a queda no desempenho.

```
from sklearn.inspection import permutation_importance

# perm_result = permutation_importance(modelo_xgb, X_test, y_test,
#                                      n_repeats=10, random_state=42)

# Exemplo de como interpretar os resultados:
# importancias_medias = perm_result.importances_mean
# for i in np.argsort(importancias_medias)[::-1][:10]:
#     subbanda = i // 9
#     descritor_idx = i % 9
#     nomes_desc = ['energia', 'rms', 'zcr', 'env_media', 'env_std',
#                   'env_kurt', 'entropia', 'fator_forma', 'fator_crista']
#     print(f"  Feature {i}: sb{subbanda:02d}_{nomes_desc[descritor_idx]} = {importancias_medias[i]:.4f}")
```

10.2.3 Atenção às Altas Frequências

Uma das razões primordiais para escolhermos a Wavelet Packet Transform (WPT) ê a sua capacidade de investigar o conteúdo espectral em alta frequência com a mesma resolução que nas baixas frequências. Artefatos de síntese costumam gerar anomalias ou vazamento de energia justamente nas faixas mais altas, detalhes muitas vezes perdidos por descritores tradicionais baseados na escala Mel.

Dica

Preste atenção especial às sub-bandas de alta frequência. Artefatos de síntese (Sistemas Text-to-Speech e Voice Conversion) costumam deixar rastros nessas faixas.

10.3 Figuras Finais

10.3.1 EER por Sub-banda / Nível / Wavelet

Gráficos de barras são excelentes para comparar o desempenho de diferentes *wavelets* mães e profundidades de decomposição.

```
import matplotlib.pyplot as plt
```

```
2
3  # Exemplo: EER por wavelet e nivel
4  wavelets = ['db4', 'db8', 'sym8']
5  niveis = [4, 5]
6
7  # Dados simulados
8  np.random.seed(42)
9  eers = np.random.rand(3, 2) * 10 + 5  # EER entre 5-15%
10
11 fig, ax = plt.subplots(figsize=(8, 5))
12 bar_width = 0.25
13 x = np.arange(len(wavelets))
14
15 for j, nivel in enumerate(niveis):
16     ax.bar(x + j*bar_width, eers[:, j], bar_width, label=f'L={nivel
       }')
17
18 ax.set_xlabel('Wavelet')
19 ax.set_ylabel('EER (%)')
20 ax.set_title('EER por Wavelet e Nivel')
21 ax.set_xticks(x + bar_width/2)
22 ax.set_xticklabels(wavelets)
23 ax.legend()
24 plt.tight_layout()
25 plt.savefig('resultados/eer_wavelet_nivel.png', dpi=150)
26 plt.show()
```

10.3.2 Curva DET / ROC

A curva ROC (Receiver Operating Characteristic) ilustra graficamente a capacidade diagnóstica de um classificador binário variando-se seu limiar.

```
1  from sklearn.metrics import roc_curve, DetCurveDisplay
2
3  # Curva ROC
4  def plotar_roc(y_true, scores, titulo='Curva ROC'):
5      fpr, tpr, _ = roc_curve(y_true, scores)
6      plt.figure(figsize=(7, 7))
7      plt.plot(fpr, tpr, 'b-', linewidth=2)
8      plt.plot([0, 1], [0, 1], 'r--')  # diagonal de chance
9      plt.xlabel('FPR (False Positive Rate)')
10     plt.ylabel('TPR (True Positive Rate)')
11     plt.title(titulo)
12     plt.grid(True)
13     plt.savefig('resultados/roc.png', dpi=150)
14     plt.show()
```

10.4 Exercícios e Desafios

Leituras do Mês

Documentação da função `permutation_importance` do `sklearn.inspection`. Artigo recomendado: Sahidullah et al. (2015), que explora recursos no domínio de alta frequência para anti-spoofing.

Exercício

1. Integre o código de EER e min-DCF e crie uma função de validação cruzada *in-domain*. 2. Compare a *Permutation Importance* com a importância de features nativa do Random Forest. 3. Crie um script para gerar as Curvas ROC de todos os seus 4 classificadores no mesmo gráfico para fins comparativos. 4. Aplique testes estatísticos para verificar se a diferença de EER entre 'db4' e 'sym8' ê significativa. 5. Faça um scatter plot dos escores preditos pelo modelo nas amostras BRSpeech-DF reais contra amostras falsas. 6. Avalie o modelo treinado com dados no idioma Português em dados em Inglês (out-of-domain) e registre o quão brusca foi a degradação (cross-corpus).

Checklist Mês 9:

- ☐ Matriz final de experimentos (todas as combinações propostas) executada e armazenada em CSV.
- ☐ Curvas ROC/DET plotadas.
- ☐ Feature Importance calculada e analisada com fôque nas sub-bandas de alta frequência.

Capítulo 11

MÊS 10 — BIBLIOTECA CONSOLIDADA + REPOSITÓRIO + REDAÇÃO (jun/2027)

Marco / Entrega

Biblioteca de extração documentada, repositório perfeitamente estruturado e o rascunho do artigo científico iniciado.

11.1 Consolidar a Biblioteca

11.1.1 Estrutura Final

Nesse ponto, os seus notebooks de experimentação já não servem para distribuição do código. É necessário migrar as funções consolidadas para um pacote em Python para facilitar reusabilidade. A estrutura ficará assim:

```
wavelet-features/
  wavelet_features/
    __init__.py
    wpt.py
    descritores.py
    extracao.py
    metricas.py
    utils.py
  tests/
    test_wpt.py
    test_descritores.py
    test_extracao.py
  pyproject.toml
  README.md
  LICENSE
```

11.1.2 O arquivo `pyproject.toml`

O padrão contemporâneo de configuração de pacotes é o arquivo `pyproject.toml`. Aqui temos um exemplo que permitirá instalar seu pacote com o clássico `pip install .`.

```
# Mostrar como criar pyproject.toml
conteudo_pyproject = '''
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.backends._legacy:_Backend"

[project]
name = "wavelet-features"
version = "0.1.0"
description = "Wavelet packet feature extraction for audio deepfake
    detection"
authors = [{name = "Thiago M. Schiavi"}]
requires-python = ">=3.9"
dependencies = [
    "numpy",
    "scipy",
    "PyWavelets",
    "soundfile",
    "scikit-learn",
]
'''
print(conteudo_pyproject)
```

11.1.3 Testes Unitários

Uma biblioteca saudável possui automação de testes. Usando a ferramenta 'pytest', podemos rodar funções que se certificam de que suas transformações continuam funcionando se o código mudar no futuro.

```
# tests/test_wpt.py
import numpy as np
import pywt

def test_reconstrucao_perfeita():
    fs = 16000
    t = np.arange(0, 1, 1/fs)
    audio = np.sin(2 * np.pi * 440 * t)

    wp = pywt.WaveletPacket(data=audio, wavelet='db4',
                            mode='symmetric', maxlevel=4)
    rec = wp.reconstruct(update=False)[:len(audio)]

    erro = np.max(np.abs(audio - rec))
    assert erro < 1e-10, f"Erro de reconstrucao muito alto: {erro}"
    print("test_reconstrucao_perfeita: PASSOU")

def test_numero_subbandas():
```

```
    audio = np.random.randn(16000)
    for nivel in [3, 4, 5]:
        wp = pywt.WaveletPacket(data=audio, wavelet='db4',
                                mode='symmetric', maxlevel=nivel)
        nodes = wp.get_level(nivel, order='freq')
        assert len(nodes) == 2**nivel, f"Esperado {2**nivel},
           obtido {len(nodes)}"
    print("test_numero_subbandas: PASSOU")

def test_conservacao_energia():
    audio = np.random.randn(16000)
    wp = pywt.WaveletPacket(data=audio, wavelet='db4',
                            mode='symmetric', maxlevel=4)
    nodes = wp.get_level(4, order='freq')

    energia_orig = np.sum(audio**2)
    energia_sub = sum(np.sum(n.data**2) for n in nodes)

    razao = energia_sub / energia_orig
    assert abs(razao - 1.0) < 0.01, f"Energia nao conservada: razao
       ={razao}"
    print("test_conservacao_energia: PASSOU")

# Executar testes
if __name__ == '__main__':
    test_reconstrucao_perfeita()
    test_numero_subbandas()
    test_conservacao_energia()
    print("\nTodos os testes passaram!")
```

11.2 Organizar o Repositório

11.2.1 Estrutura Completa do Projeto de Pesquisa

Não confunda o diretório da *biblioteca* com o do *projeto de pesquisa* (a pipeline completa do PIBITI). Este precisa organizar também os datasets, os features já extraídos, protocolos e resultados.

```
pibiti-deepfake/
  data/              # NAO versionar audios (LGPD/licencas)
  splits/            # arquivo de particao (versionado)
  src/               # codigo principal do experimento
  features/          # features extraidas (.npy)
  results/           # CSVs de resultados, figuras
  reports/           # relatorio parcial, final
  notebooks/         # analises exploratorias
  wavelet_features/  # biblioteca reutilizavel (submodulo)
  tests/             # testes unitarios
  requirements.txt
```

```
environment.yml    # conda (opcional)
README.md
.gitignore
```

Atenção

NÃO redistribua àudios no repositório do GitHub. Versionar dados de mídia ê ruim para o sistema (git) e muito provavelmente irá violar políticas de privacidade (LGPD) e licenças dos autores originais. Versionar somente código, scripts, protocolos, e resultados analíticos agregados.

11.2.2 README.md Completo

Seu documento README deve servir de porta de entrada. Inclua um cabeçalho descritivo, os pré-requisitos de instalação, a forma de uso básica, alguns gráficos ou EERs que você conseguiu alcançar e um modelo para citação acadêmica.

11.3 Rascunho do Artigo

Agora que o trabalho técnico está bem fundamentado, é hora de escrever a pesquisa, adaptando a apresentação para a formatação exigida em congressos nacionais. Sugestão de estrutura para ENIAC / CONIC / SEMESP / REIC:

- **Título** e **Resumo/Abstract**
- **Introdução** (Contexto, problema e objetivos)
- **Trabalhos Relacionados** (estado da arte de deepfake em aúdio)
- **Metodologia** (Extração via WPT + descritores + família de classificadores)
- **Experimentos** (Apresentação dos corpora, divisões de validação e métricas - EER/DCF)
- **Resultados e Discussão** (Apresentação e análise da importância das *features*)
- **Conclusão**
- **Referências**

11.4 Exercícios e Desafios

Leituras do Mês

Verifique as normas de formatação do ENIAC ou templates institucionais de IC (ex. CONIC). Leia também o "ML Reproducibility Checklist"e a documentação do sistema build `pyproject.toml`.

Exercício

1. Complete e rode todos os testes no script `test_wpt.py`. Implemente testes extras para os seus descritores. 2. Crie o seu arquivo `.gitignore` apropriado, ignorando as pastas de datasets, modelos pesados (HDF5/PKL) e os '.npy' das features extraídas. 3. Escreva a secção de Metodologia do artigo, deixando-a o mais detalhada possível para que sua proposta (os 9 descritores, famílias de wavelets e a escolha do classificador) possa ser facilmente reproduzida por leitores. 4. Esboce a *Tabela de Resultados Principais* do artigo baseada nas saídas dos CSVs gerados neste capítulo.

Checklist Mês 10:

☐ Biblioteca `wavelet_features` criada e devidamente organizada.

☐ Arquivo de configuração (`pyproject.toml`) criado.

☐ Scripts de teste e repositório principal configurados (.gitignore, README).

☐ Rascunho do artigo iniciado.

Parte VI

Finalização

Capítulo 12

MÊS 11 — SUBMISSÃO DO ARTIGO + FINALIZAÇÃO DO ARTEFATO (jul/2027)

Marco / Entrega

Artigo submetido + biblioteca/repositório públicos e reprodutíveis.

12.1 Revisão final do artigo

12.1.1 Checklist de revisão

Antes de enviar o seu artigo, certifique-se de que todos os itens essenciais foram contemplados. Siga este checklist para uma submissão tranquila:

- ☐ Título claro e descritivo
- ☐ Resumo com objetivo, método e resultados principais
- ☐ Introdução com motivação e contribuição
- ☐ Metodologia reprodutível (parâmetros explicitados)
- ☐ Resultados com EER e min-DCF por corpus/wavelet/classificador
- ☐ Discussão com análise de sub-bandas discriminantes
- ☐ Conclusão com limitações e trabalhos futuros
- ☐ Referências completas e formatadas
- ☐ Figuras em alta resolução (300 dpi)
- ☐ Tabelas legíveis

12.1.2 Template de tabela de resultados

Aqui está um código simples para gerar tabelas em LaTeX diretamente dos seus resultados em Python, facilitando a vida na hora de redigir o artigo:

```python
# Gerar tabela LaTeX automaticamente
def gerar_tabela_latex(resultados):
    linhas = []
    linhas.append('\\begin{tabular}{llllrr}')
    linhas.append('\\toprule')
    linhas.append('Corpus & Wavelet & N\\'ivel & Classif. & EER (\\%) & min-DCF \\\\')
    linhas.append('\\midrule')

    for r in resultados:
        linha = f"{r['corpus']} & {r['wavelet']} & {r['nivel']} & {r['clf']} & {r['eer']:.2f} & {r['min_dcf']:.4f} \\\\"
        linhas.append(linha)

    linhas.append('\\bottomrule')
    linhas.append('\\end{tabular}')

    return '\n'.join(linhas)

# Exemplo
resultados_exemplo = [
    {'corpus': 'BRSpeech-DF', 'wavelet': 'db4', 'nivel': 4, 'clf': 'XGBoost', 'eer': 5.23, 'min_dcf': 0.1234},
    {'corpus': 'BRSpeech-DF', 'wavelet': 'db4', 'nivel': 4, 'clf': 'GMM', 'eer': 8.45, 'min_dcf': 0.2156},
    {'corpus': 'BraSpoof-50', 'wavelet': 'db4', 'nivel': 4, 'clf': 'XGBoost', 'eer': 6.78, 'min_dcf': 0.1567},
]

tabela = gerar_tabela_latex(resultados_exemplo)
print(tabela)
```

12.2 Eventos alvo

Avalie onde publicar seus resultados. Aqui estão algumas sugestões:

- **ENIAC (Encontro Nacional de Inteligência Artificial e Computacional):** Excelente congresso nacional para publicar pesquisas aplicadas de IA.
- **CONIC/SEMESP:** Congresso focado em Iniciação Científica, ótimo para apresentar o panorama completo do seu PIBITI.
- **REIC (Revista Eletrônica de Iniciação Científica):** Uma opção caso deseje publicar um artigo mais extenso em um periódico nacional.

Dica

Verifique os prazos e formatos de cada evento. Prepare o artigo no template do evento alvo.

12.3 Congelar a biblioteca e o repositório

12.3.1 Versionamento final

Após concluir o código usado para gerar os resultados do artigo, crie uma *tag* no Git:

```
git tag -a v1.0 -m "Versao final - submissao do artigo"
git push origin v1.0
```

12.3.2 LICENSE

Adicione um arquivo de licença ao seu repositório para permitir que outros usem sua biblioteca:

```
# Conteudo do arquivo LICENSE (MIT)
licenca_mit = '''
MIT License

Copyright (c) 2027 Thiago M. Schiavi

Permission is hereby granted, free of charge, to any person
    obtaining a copy
of this software and associated documentation files (the "Software
    "), to deal
in the Software without restriction, including without limitation
    the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/
    or sell
copies of the Software, and to permit persons to whom the Software
    is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be
    included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
    EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
    MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT
    SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR
    OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
    ARISING FROM,
```

```
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
    DEALINGS IN THE
SOFTWARE.
'''
print(licenca_mit)
```

12.3.3 CITATION.cff

Facilite a citação do seu software pelos outros pesquisadores criando um arquivo `CITATION.cff`:

```
citation_cff = '''
cff-version: 1.2.0
title: "Wavelet Packet Feature Extraction for Audio Deepfake
    Detection"
message: "If you use this software, please cite it as below."
authors:
  - family-names: Schiavi
    given-names: Thiago Mateus
    affiliation: Fatec Ourinhos
type: software
version: "1.0.0"
date-released: "2027-07-01"
repository-code: "https://github.com/thiago-schiavi/pibiti-deepfake
    "
license: MIT
'''
print(citation_cff)
```

12.3.4 DOI via Zenodo (opcional)

O Zenodo atribui um DOI (Digital Object Identifier) ao seu *release* do GitHub, tornando o seu software uma publicação oficial. Passos:

1. Conecte o GitHub ao Zenodo.
2. Crie um *release* no GitHub.
3. O Zenodo arquiva automaticamente o repositório e atribui um DOI.

12.4 Licenças dos corpora e LGPD

Atenção

Confira as licenças de cada corpus e dos modelos de síntese. Restrinja o uso a fins acadêmicos. Os áudios contêm dados pessoais (voz) — observe a LGPD. NÃO redistribua áudios no repositório.

```python
# Checklist de conformidade
conformidade = {
    'BRSpeech-DF': {'licenca': 'verificar', 'redistribuicao': 'NAO'},
    'FakeBrAccent': {'licenca': 'verificar', 'redistribuicao': 'NAO'},
    'Fake Voices': {'licenca': 'verificar', 'redistribuicao': 'NAO'},
    'BraSpoof-50': {'licenca': 'verificar', 'redistribuicao': 'NAO'},
}

print(f"{'Corpus':20s} {'Licenca':15s} {'Redistribuir':>15s}")
print('-' * 55)
for nome, info in conformidade.items():
    print(f"{nome:20s} {info['licenca']:15s} {info['redistribuicao']:>15s}")
```

12.5 Exercícios

Exercício

Configure a integração do seu repositório GitHub com o Zenodo e emita um DOI de teste.

Exercício

Crie a estrutura da sua *release* final no GitHub, garantindo que o `README.md`, `LICENSE` e `CITATION.cff` estejam lá.

Exercício

Escreva uma seção "Ethics Statement"ou "Termos de Uso"no seu repositório declarando o uso responsável do código e os cuidados com dados biométricos.

Leituras do Mês

Documentação de licenciamento dos corpora e de arquivamento com DOI (Zenodo/CITATION.cff).

Checklist Month 11:

- ☐ Artigo revisado e submetido
- ☐ Biblioteca congelada (tag v1.0)
- ☐ LICENSE adicionado
- ☐ CITATION.cff criado
- ☐ DOI gerado (opcional)

☐ Licenças dos corpora verificadas

☐ LGPD observada

Capítulo 13

MÊS 12 — RELATÓRIO FINAL + ENCERRAMENTO (ago/2027)

Marco / Entrega

Relatório final entregue + apresentação pronta + biblioteca de features wavelet publicada.

13.1 Relatório final

13.1.1 Estrutura do relatório

Seu relatório final de PIBITI deve consolidar todo o ano de pesquisa. Eis um template sugerido:

1. **Capa**
2. **Resumo**
3. **Introdução:** objetivo, justificativa e o problema dos deepfakes.
4. **Referencial Teórico:** wavelets, WPT, descritores de áudio, anti-spoofing.
5. **Metodologia:** pipeline completa, descrição dos corpora, e métricas (EER, min-DCF).
6. **Resultados:**
   - Por corpus/nível/wavelet
   - Análise de sub-bandas discriminantes
   - Custo em CPU
7. **Discussão:**
   - Comparação com baselines cepstrais (se disponível)
   - Limitações encontradas
8. **Conclusão e Trabalhos Futuros:**

- Avaliação cross-dataset
- Fusão com descritores cepstrais

9. **Referências**
10. **Apêndices:** link para o código, tabelas completas, figuras adicionais.

13.1.2 Compilar resultados finais

```
import csv
import numpy as np

def compilar_resultados(caminho_csv):
    with open(caminho_csv, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        resultados = list(reader)

    # Melhor resultado por corpus
    por_corpus = {}
    for r in resultados:
        corpus = r['corpus']
        eer = float(r['eer'])
        if corpus not in por_corpus or eer < float(por_corpus[
            corpus]['eer']):
             por_corpus[corpus] = r

    print("MELHORES RESULTADOS POR CORPUS:")
    print(f"{'Corpus':20s} {'Wavelet':8s} {'Nivel':>6s} {'Classif
        ':>10s} {'EER':>8s} {'min-DCF':>10s}")
    print('-' * 70)
    for corpus, r in por_corpus.items():
        print(f"{corpus:20s} {r['wavelet']:8s} {r['nivel']:>6s} {r
            ['classificador']:>10s} {r['eer']:>8s} {r['min_dcf']:>10
            s}")
```

13.1.3 Script para gerar todas as figuras finais

```
import matplotlib.pyplot as plt
import numpy as np

def gerar_figuras_finais():
    # Figura 1: EER por configuracao
    # Figura 2: Importancia por sub-banda
    # Figura 3: Curvas DET
    # Figura 4: Custo CPU por configuracao

    print("Figuras geradas em results/figuras/")
    # ... (implementar conforme resultados reais)

gerar_figuras_finais()
```

13.2 Apresentação / Pôster

13.2.1 Estrutura da apresentação

- **Slide 1:** Título e autores
- **Slide 2:** Motivação (deepfakes de áudio)
- **Slide 3:** Objetivo
- **Slide 4:** Pipeline (diagrama visual)
- **Slide 5:** WPT + Descritores escolhidos
- **Slide 6-7:** Resultados principais (tabelas, gráficos DET)
- **Slide 8:** Análise de sub-bandas (o que importou mais?)
- **Slide 9:** Conclusão e trabalhos futuros

Dica

Prepare a apresentação com foco nos RESULTADOS. O público quer saber: funciona? O que discrimina? Quanto custa?

13.3 Entregas finais

```
# Checklist de entrega final
entregas = [
    ('Relatorio final (PDF)', 'reports/relatorio_final.pdf'),
    ('Apresentacao/poster', 'reports/apresentacao.pdf'),
    ('Biblioteca de features', 'wavelet_features/'),
    ('Resultados por corpus', 'results/resultados_completos.csv'),
    ('Codigo-fonte', 'src/'),
    ('Testes', 'tests/'),
    ('Arquivo de particao', 'splits/particao.csv'),
    ('README.md', 'README.md'),
    ('requirements.txt', 'requirements.txt'),
    ('Link do artigo/preprint', 'reports/artigo_submetido.pdf'),
]

print("ENTREGAS FINAIS:")
print(f"{'Item':40s} {'Local':40s} {'Status'}")
print('-' * 90)
for item, local in entregas:
    print(f"{item:40s} {local:40s} {'[ ]'}")
```

13.4 Trabalhos futuros

Ao finalizar o projeto, deixe claro as próximas etapas possíveis para a pesquisa:

- **Avaliação cross-dataset:** Treinar em um corpus e testar em outro, conforme discutido por Müller et al. (2022).
- **Fusão wavelet + cepstral:** Combinar as features extraídas via WPT com clássicos como LFCC ou MFCC.
- **Deep learning sobre sub-bandas:** Usar CNNs ou Transformers diretamente sobre os coeficientes da WPT.
- **Expansão para outros idiomas:** Testar as técnicas em bases internacionais para avaliar generalização.

Leituras do Mês

Müller et al. (2022) — Does Audio Deepfake Detection Generalize?

Checklist Month 12:

☐ Relatório final redigido

☐ Apresentação pronta

☐ Biblioteca entregue

☐ Resultados compilados

☐ Link do artigo/preprint

☐ Repositório organizado e público

Parte VII

Apêndices

Capítulo 14

CHECKLIST GERAL DO PROJETO

Abaixo está um checklist macro para todo o projeto PIBITI de 12 meses:

Mês 1:

☐ Repositório Git criado

☐ Ambiente Python funcionando

☐ Corpora inventariados

☐ Wavelets/WPT estudadas

Mês 2:

☐ Partição validada/gerada

☐ Arquivo congelado

☐ WPT teoria consolidada

Mês 3:

☐ Referencial teórico (v1)

☐ WPT decomposição funcionando

☐ Reconstrução validada

☐ Testes sintéticos OK

Mês 4:

☐ Nível/wavelet do núcleo definidos

☐ Energia/RMS implementados

☐ ZCR implementado

☐ Envelope (Hilbert) iniciado

Mês 5:

☐ Todos os descritores implementados

☐ Matriz de features gerada

☐ Sanity checks OK

☐ PCA/t-SNE exploratório

Mês 6:

☐ GMM treinado

☐ XGBoost treinado

☐ EER/min-DCF calculados

☐ Biblioteca v0 empacotada

☐ Relatório parcial entregue

Mês 7:

☐ SVM implementado

☐ Random Forest implementado

☐ db8/sym8 testadas

☐ L=5 testado

Mês 8:

☐ Novos corpora incorporados

☐ Custo CPU medido

Mês 9:

☐ Avaliação completa

☐ Feature importance

☐ Figuras finais

Mês 10:

☐ Biblioteca consolidada

☐ Repositório organizado

☐ Rascunho do artigo

Mês 11:

☐ Artigo submetido

☐ LICENSE/CITATION.cff

☐ Tag v1.0

Mês 12:

☐ Relatório final entregue

☐ Apresentação pronta

☐ Biblioteca publicada

Capítulo 15

BOAS PRÁTICAS TRANSVERSAIS

Estas práticas devem guiar seu desenvolvimento do início ao fim do projeto:

1. **Valide a reconstrução da WPT antes de tudo:** O sinal original deve ser recuperável. Isso garante que sua árvore de decomposição faz sentido.
2. **Partição única e congelada:** Defina quem é Treino, Validação e Teste logo no início e não mude mais. Evita vazamento de dados.
3. **Extraia features uma vez e reuse (cache):** O cálculo da WPT é pesado. Salve os descritores em disco (`.npy` ou `.csv`) e carregue na hora de treinar os modelos.
4. **Meça o custo em CPU desde o início:** O diferencial da sua solução é a leveza. Mantenha os tempos de processamento rastreados.
5. **Versionar tudo em Git:** Commits frequentes salvam vidas. Código quebrado sempre pode ser revertido.
6. **Não misturar validação e teste:** O conjunto de teste deve ser intocado até o final, servindo apenas para a avaliação final do modelo escolhido.
7. **Priorizar EER/min-DCF (não acurácia):** Acurácia engana em bases desbalanceadas. EER e min-DCF são o padrão da indústria biométrica.
8. **Declarar parâmetros do min-DCF fixos (Cmiss=Cfa=1, Ptar=0.05):** Garante comparações justas com outros trabalhos.
9. **Reuniões quinzenais com o orientador:** Mantenha contato constante. Anote dúvidas e mostre os progressos.
10. **Escrever o relatório em paralelo:** Não deixe a escrita para o último mês. Documente resultados assim que eles saírem.

Capítulo 16

GLOSSÁRIO

Amplitude: intensidade do sinal em um ponto.

Bonafide: fala real, genuína.

Corpus (pl. corpora): base de dados de áudio.

DWT: Discrete Wavelet Transform.

EER: Equal Error Rate. Ponto onde a taxa de falsa aceitação e falsa rejeição são iguais.

Envelope: contorno de amplitude do sinal.

FAR: False Acceptance Rate.

Feature: característica numérica extraída do sinal.

FRR: False Rejection Rate.

GMM: Gaussian Mixture Model.

Hilbert (transformada de): usada para obter o envelope do sinal de forma analítica.

In-domain: avaliação no mesmo domínio/corpus em que o modelo foi treinado.

min-DCF: minimum Detection Cost Function.

Mono: áudio com 1 canal.

Nyquist: metade da taxa de amostragem (frequência máxima representável).

Padding: preenchimento nas bordas do sinal para adequar o tamanho durante operações de convolução.

RMS: Root Mean Square. Medida de energia efetiva do sinal.

Sample rate (taxa de amostragem): amostras por segundo (Hz).

Spoof: fala sintética/falsificada.

Sub-banda: faixa de frequência resultante da decomposição WPT.

TTS: Text-to-Speech.

VC: Voice Conversion.

Wavelet: função oscilante localizada usada para análise.

Wavelet-mãe (mother wavelet): wavelet base (ex: db4).

WPT: Wavelet Packet Transform.

XGBoost: eXtreme Gradient Boosting.

ZCR: Zero Crossing Rate.

Capítulo 17

BIBLIOGRAFIA RECOMENDADA

Fundamentos wavelet/wavelet packet:

- **Daubechies, I. (1990):** O texto base que popularizou a teoria das wavelets ortogonais compactas.
- **Mallat, S. (2009):** "A Wavelet Tour of Signal Processing-- a bíblia do processamento de sinais com wavelets.
- **Coifman, R.; Wickerhauser, M. (1992):** Autores clássicos sobre entropia e seleção da melhor base (Best Basis) para pacotes wavelet.
- **Lee, G. R. et al. (2019):** Artigo do PyWavelets, a biblioteca Python essencial para WPT.

Descritores e ferramentas de sinal:

- **scipy.signal.hilbert:** Documentação oficial para cálculo do envelope via Hilbert.
- **scipy.stats:** Para estatísticas de ordem superior (skewness, kurtosis).
- **Definições de fator de forma e fator de crista:** Em qualquer livro de processamento ou documentação de engenharia.

Anti-spoofing:

- **Wang, X. et al. (2020) - ASVspoof 2019:** Artigo descritivo do desafio mundial clássico de anti-spoofing, essencial para entender métricas.
- **Sahidullah, M. et al. (2015):** Comparação detalhada de descritores cepstrais, fundamental para baselines.
- **Mueller, N. et al. (2022):** Excelente leitura sobre o desafio da generalização (cross-dataset) em modelos de detecção.

Corpora PT-BR:

- **Costa Ferro Filho et al. (2025) - BRSpeech-DF:** Corpus com áudios autênticos e sintéticos em português para benchmark.
- **Santos et al. (2025) - FakeBrAccent:** Focado em variações de sotaque em amostras falsificadas.

- **UNFAKE (2024) - Fake Voices:** Dataset com vózes sintetizadas variadas e cenários reais.
- **Franca & Almeida (2026) - BraSpoof-50:** Recente corpus com alta diversidade de falantes em português.

Aprendizado de máquina:

- **Chen & Guestrin (2016) - XGBoost:** Artigo oficial do método de gradient boosting mais usado na indústria.
- **Pedregosa et al. (2011) - scikit-learn:** A biblioteca base para todo seu pipeline de aprendizado clássico.
- **Reynolds - GMM:** Qualquer referência a Douglas Reynolds explica bem os modelos de mistura gaussiana para áudio.

Dica

Parabéns por completar todo o guia! Você agora tem um mapa completo de 12 meses para o seu PIBITI. Bom trabalho e boa pesquisa!